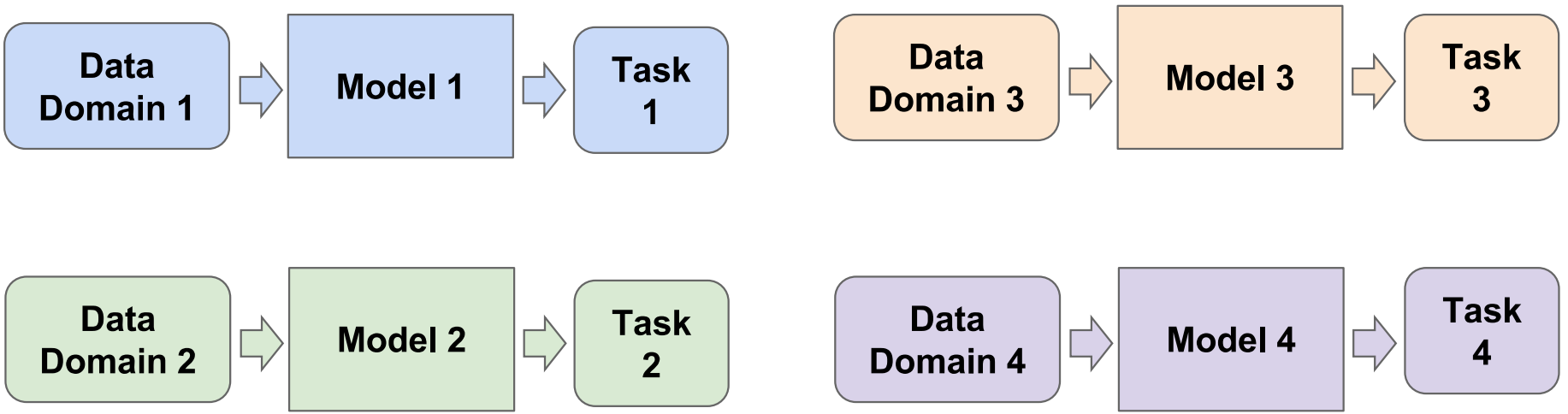
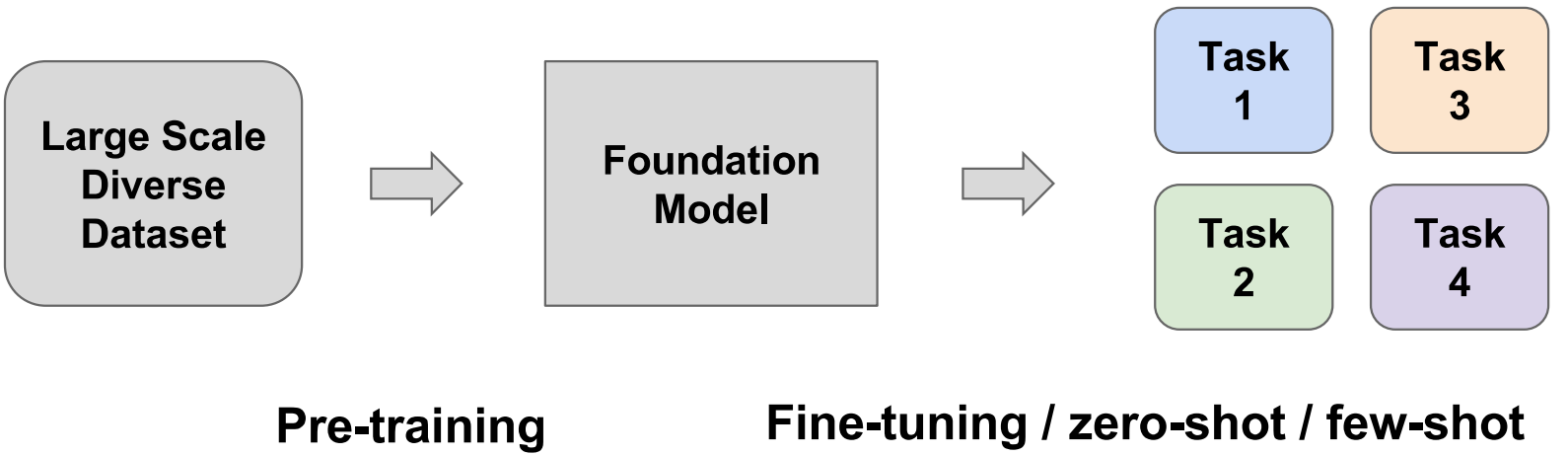
How have we been thinking about models in this class so far?

Train a *specialized* model for *each* task



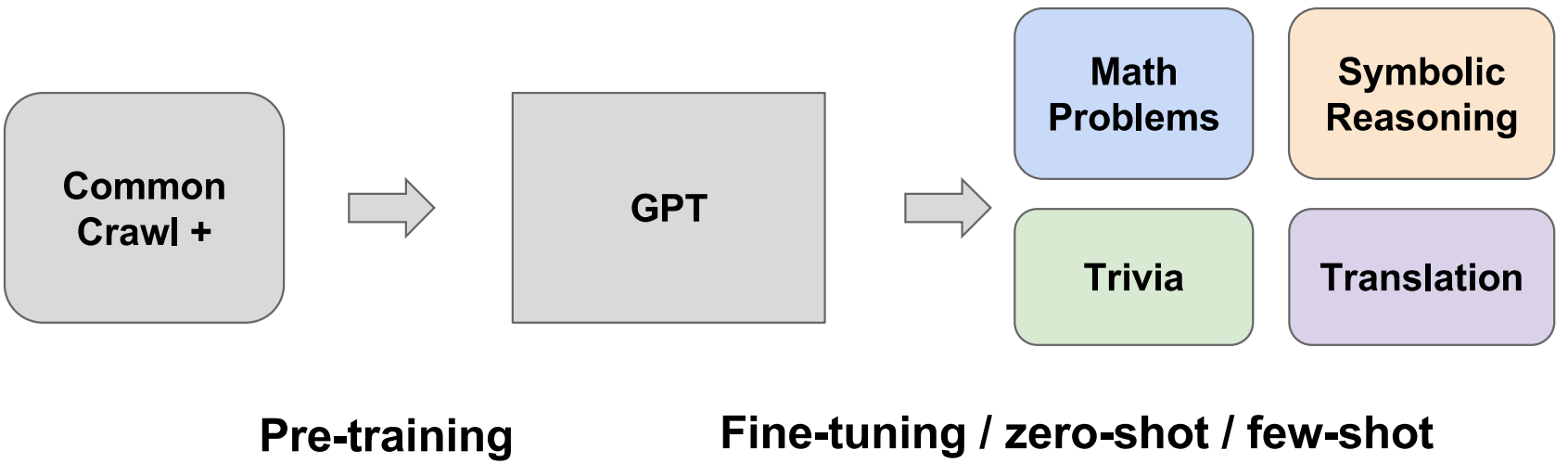
Now, we build Foundation Models

Pre-train one model that acts as the *foundation* for many different tasks



Foundation Models

Language



There are many classes of Foundation Models

Language	Classification	LM + Vision	And More!	Chaining
ELMo	CLIP	LLaVA	Segment Anything	LMs + CLIP
BERT	CoCa	Flamingo	Whisper	Visual Programming
GPT		GPT-4V	Dalle	
T5		Gemini	Stable Diffusion	
		Molmo	Imagen	

How do identify a model as a Foundation?

Always see with foundation models:

- general /robust to many different tasks

Often see with foundation models:

- Large # params
- Large amount of data
- Self-supervised pre-training objective

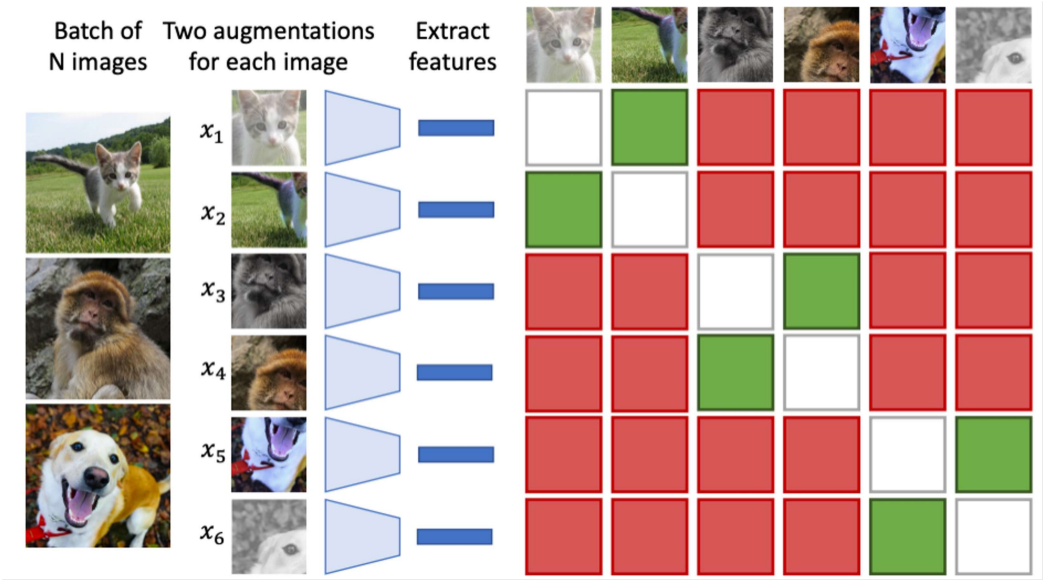
Language models are out of scope for this class

Language	Classification	LM + Vision	And More!	Chaining
ELMo	CLIP	LLaVA	Segment Anything	LMs + CLIP
BERT	CoCa	Flamingo	Whisper	Visual Programming
GPT		GPT-4V	Dalle	
T5		Gemini	Stable Diffusion	
		Molmo	Imagen	

Let’s start with the foundation models for classification

Language	Classification	LM + Vision	And More!	Chaining
ELMo	**CLIP**	**LLaVA**	**Segment Anything**	**LMs + CLIP**
BERT	**CoCa**	**Flamingo**	Whisper	**Visual Programming**
GPT		GPT-4V	Dalle	
T5		Gemini	Stable Diffusion	
		Molmo	Imagen	

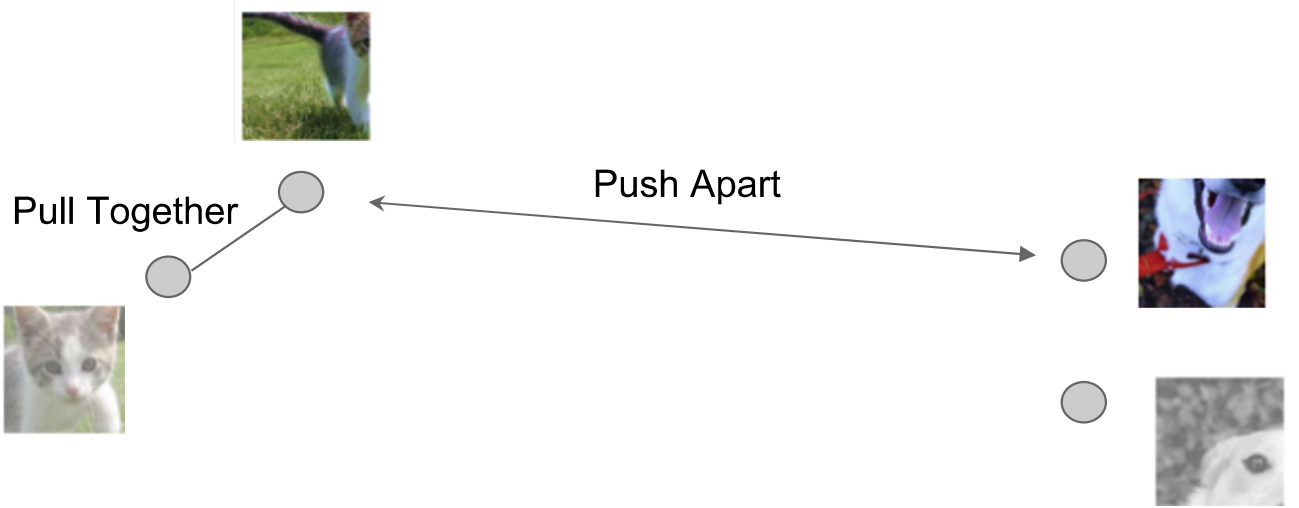
Recall this self-supervised objective from SimCLR



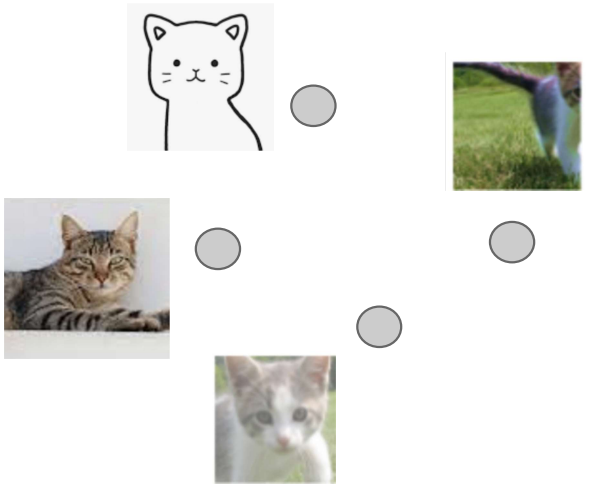
Use Self Supervised learning to learn good image features

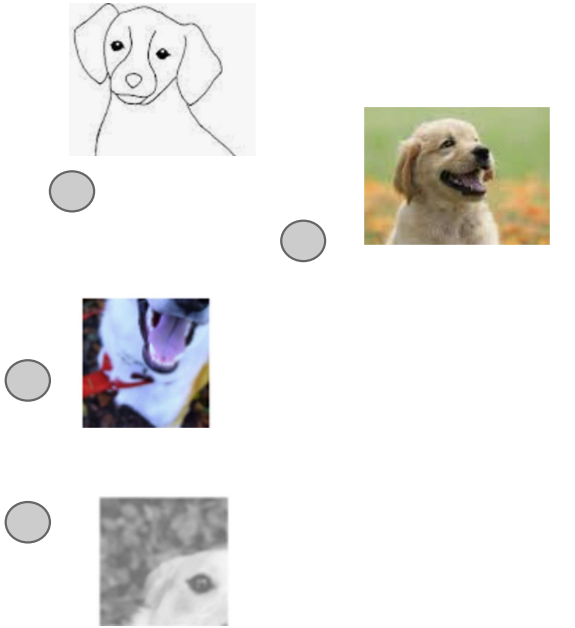
Can train small classifiers on top of these features using supervised learning

The main idea was to learning concepts without labels -> a self-supervised pretraining objective

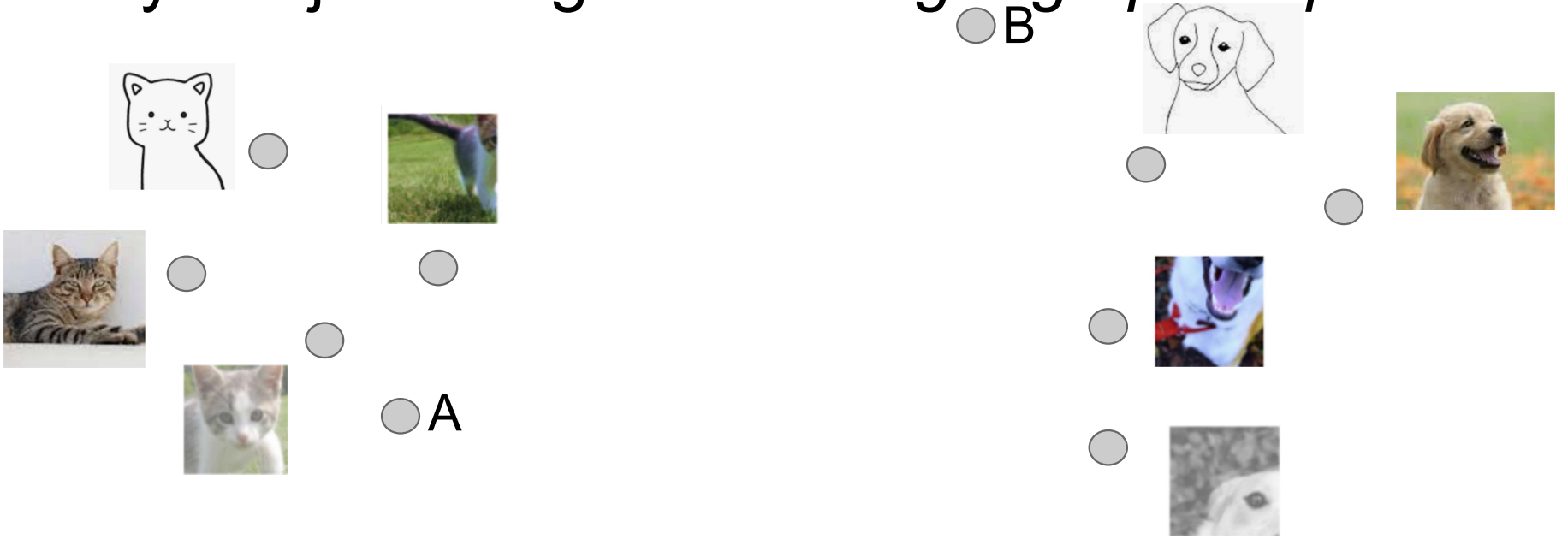


The hope was that the learned representations generalize to new instances





Can we generalize these representations beyond just images? *To language perhaps?*



1. “A cute fluffy cat”
2. “My favorite dog is a golden retriever”

What if this representation space could also embed sentences/phrases?









“A cute fluffy cat”

“My favorite dog is a golden retriever”

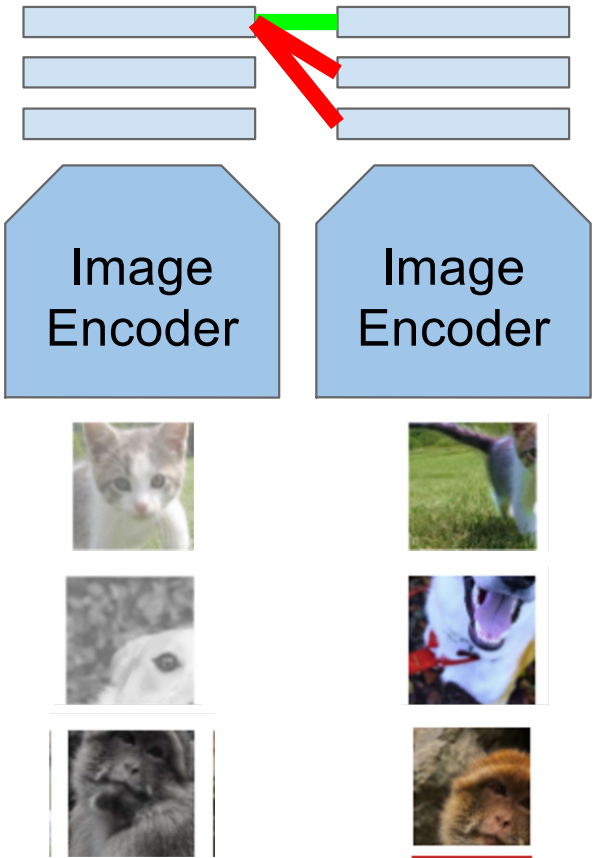




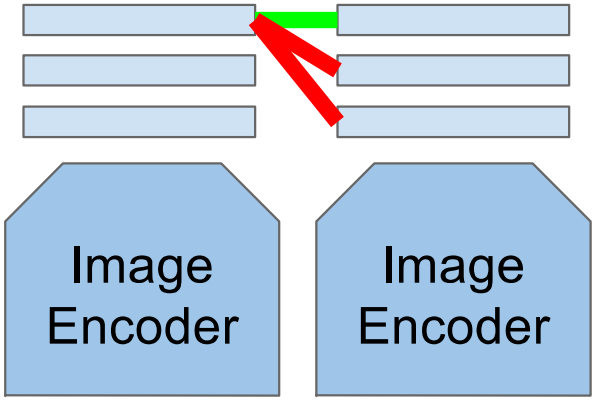




SimClr

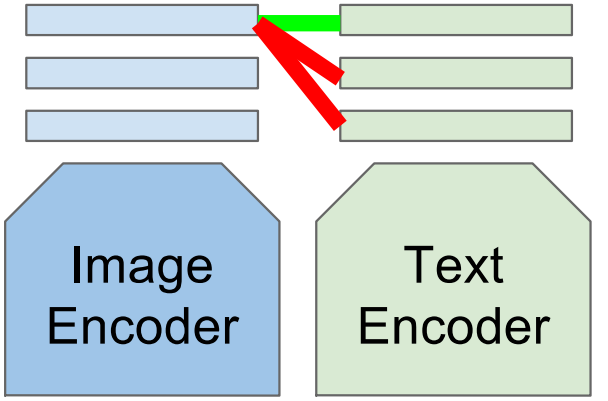


SimClr





CLIP





“My favorite dog is a golden retriever”



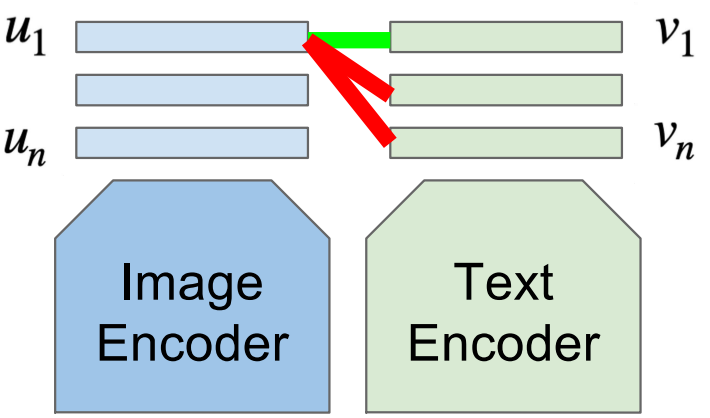
“A cute fluffy cat”



“Monkeys are my favorite animal”

CLIP is trained with the same contrastive objective

$$\sum_{i=1}^{n} -\log\left(\frac{e^{\langle u_i, v_i\rangle}}{\sum_{j=1}^{n} e^{\langle u_i, v_j\rangle}}\right)$$





"My favorite dog is a golden retriever"



"A cute fluffy cat"

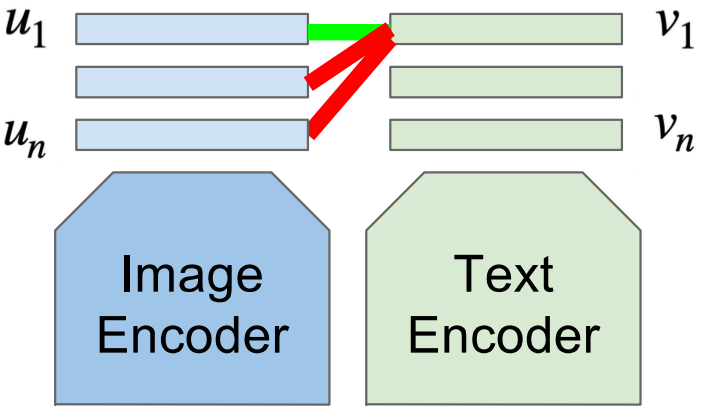


"Monkeys are my favorite animal"

CLIP Training Objective

$$\sum_{i=1}^{n} -\log\left(\frac{e^{\langle u_i, v_i\rangle}}{\sum_{j=1}^{n} e^{\langle u_i, v_j\rangle}}\right)$$

$$+\sum_{i=1}^{n} -\log\left(\frac{e^{\langle u_i, v_i\rangle}}{\sum_{j=1}^{n} e^{\langle u_j, v_i\rangle}}\right)$$





"My favorite dog is a golden retriever"



"A cute fluffy cat"



"Monkeys are my favorite animal"

Lots of image-text data can be found online

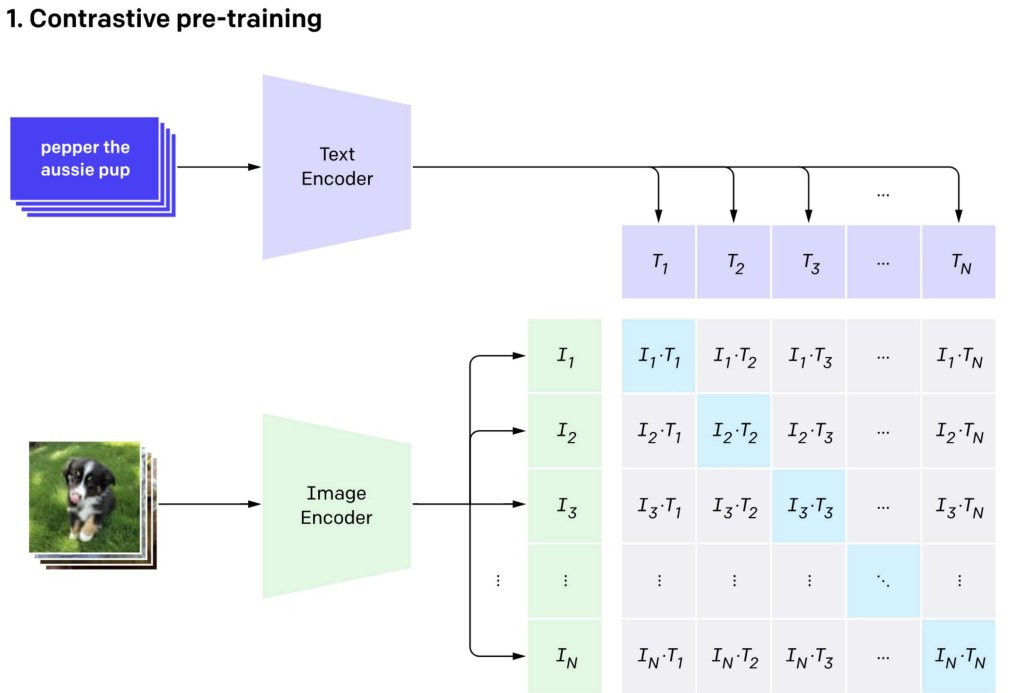


Mount Rainier's northwestern slope viewed aerially just before sunset on September 6, 2020

CLIP training data was scraped at scale from images and their associated alt-text from the internet

https://en.wikipedia.org/wiki/Mount_Rainier

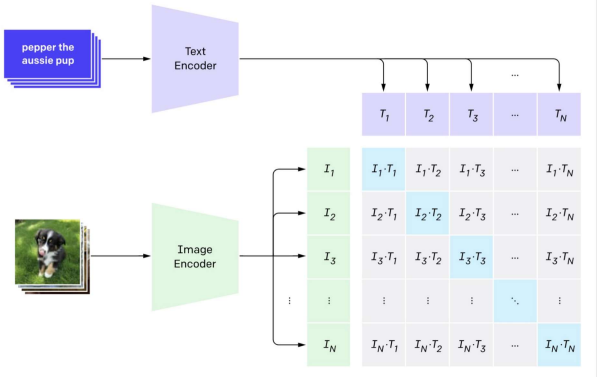
CLIP Training Objective



At the end of training, you have a model that will give you a similarity score between an image and a text

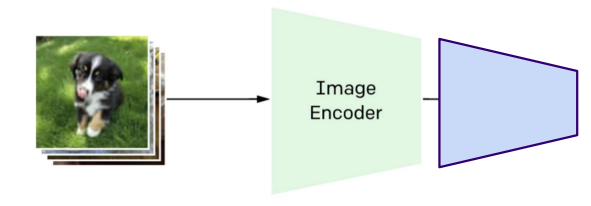
Using pre-trained models out of the box

Step 1: Pretrain a network on a pretext task that doesn't require supervision



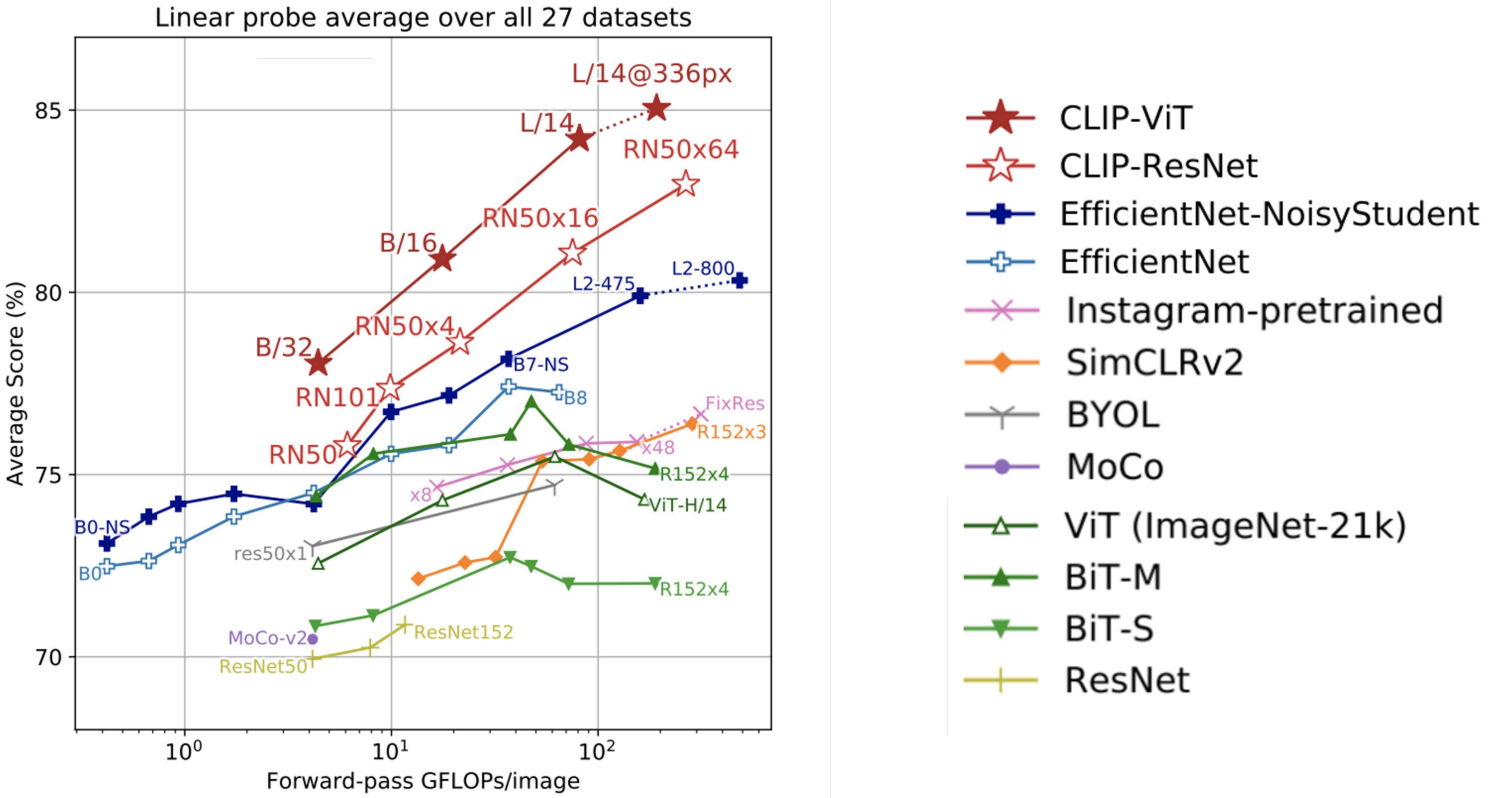
Pre-training tasks:
Contrastive Objective

Step 2: Transfer encoder to downstream tasks via linear classifiers



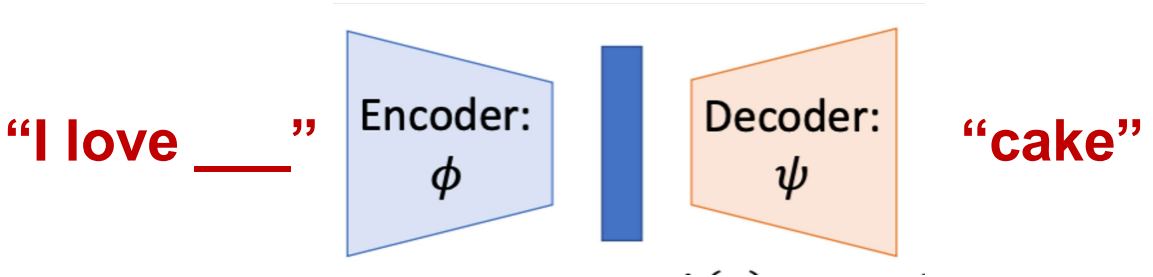
Downstream tasks:
Image classification, object detection, semantic segmentation

CLIP features w/ linear probe across multiple datasets

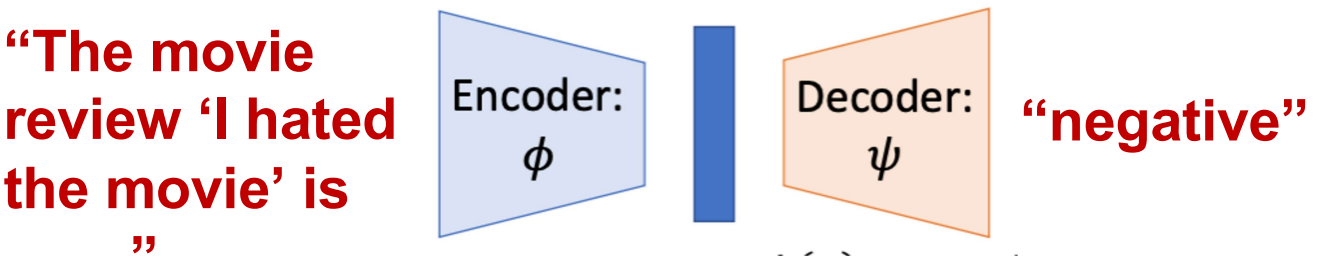


Big difference with language models: We can use LLMs zero-shot for new downstream tasks

Step 1: Pretrain a network on a pretext task that doesn't require supervision

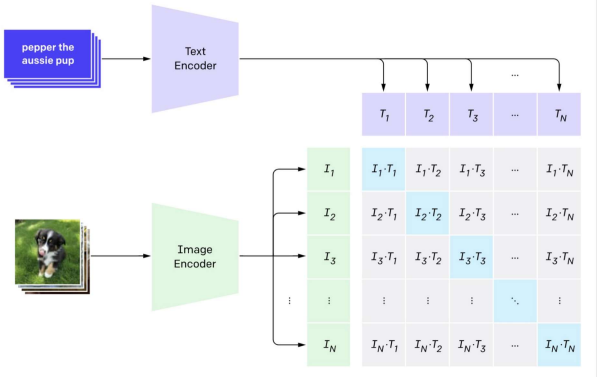


Step 2: Use the model out of the box in a creative way!



But how do we use pre-trained vision-language models in a zero-shot manner?

Step 1: Pretrain a network on a pretext task that doesn't require supervision

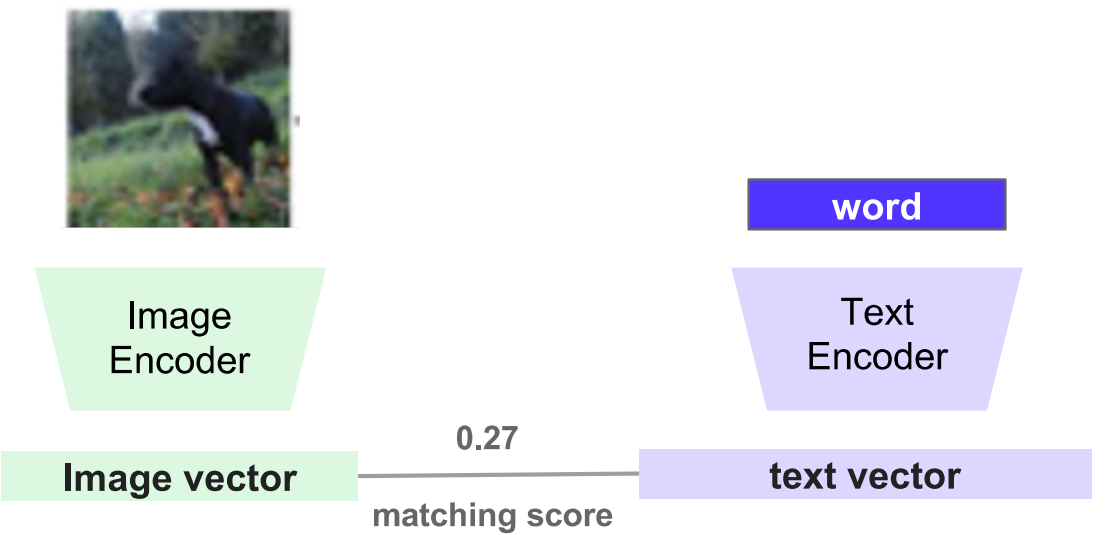


Pre-training tasks:
Contrastive Objective

Step 2: Use the model out of the box in a creative way!

Out of the box classification (No fine-tuning)

Clever trick: we can create a classifier using the text encoder!



Radford et al “Learning Transferable Visual Models From Natural Language Supervision”

Create a vector representation for *each* category!



Image Encoder

Image vector

plane

dog

bird

Text Encoder

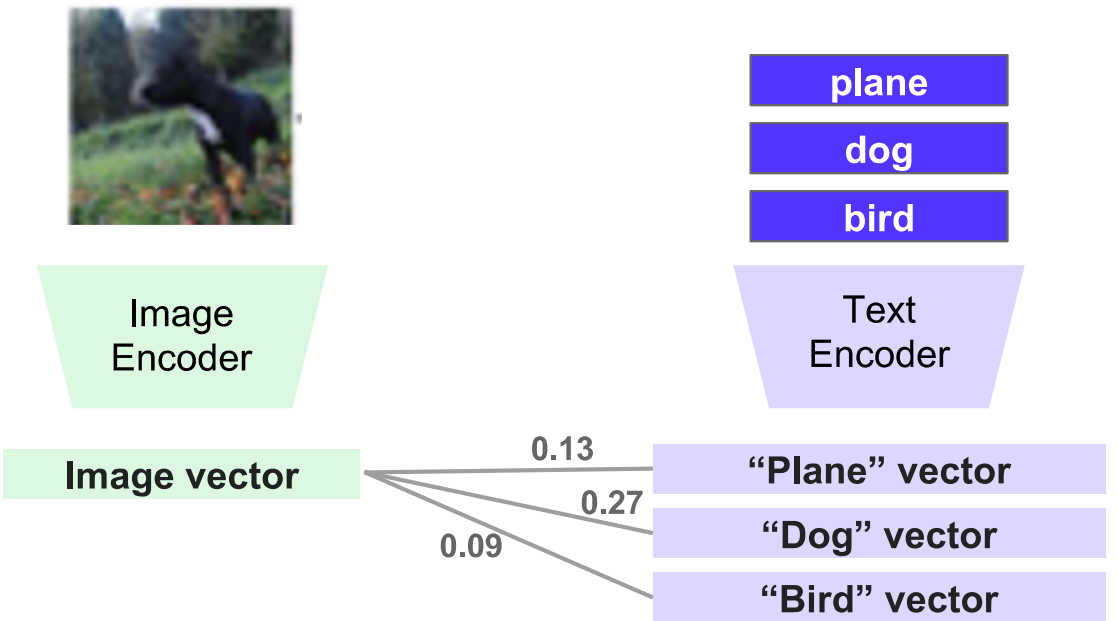
“Plane” vector

“Dog” vector

“Bird” vector

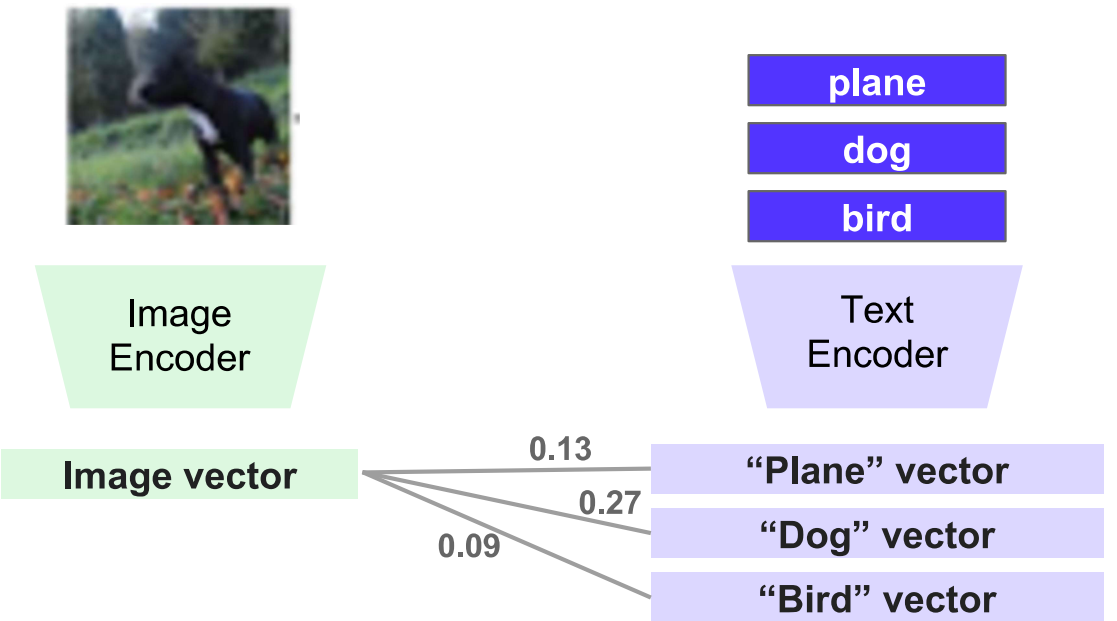
Radford et al “Learning Transferable Visual Models From Natural Language Supervision”

Match a new image to the most similar vector



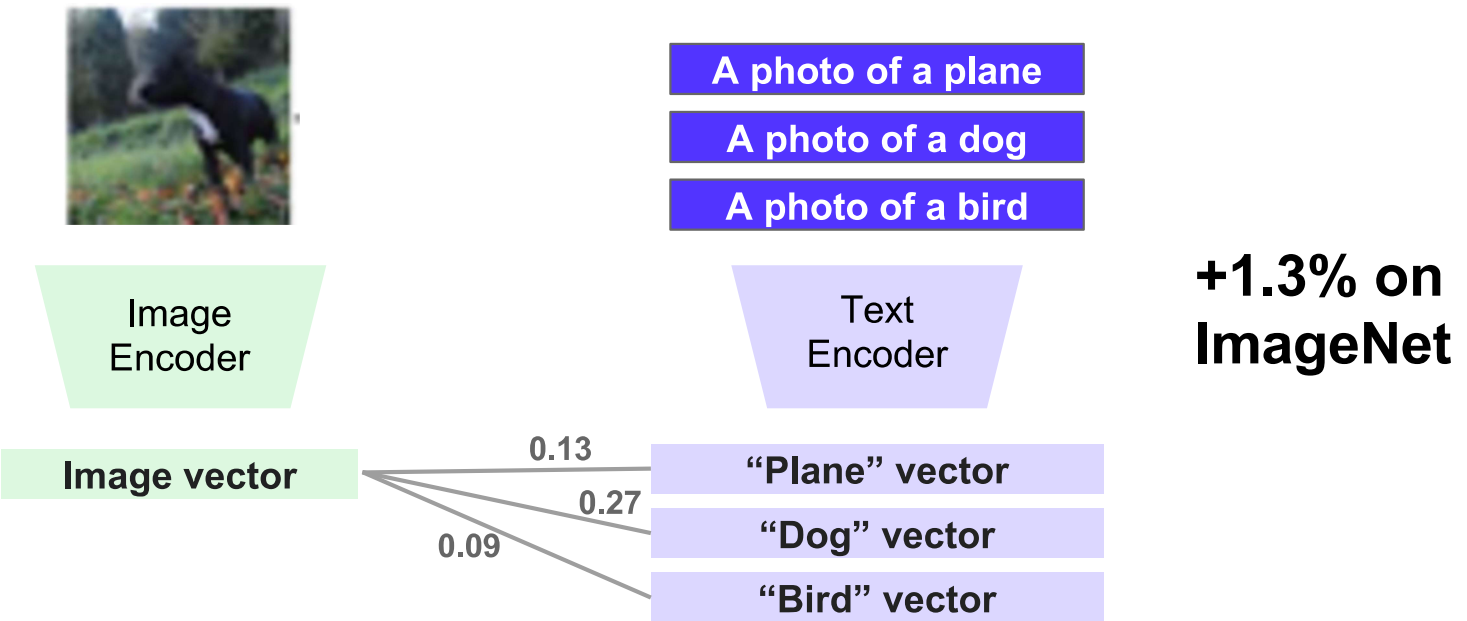
Radford et al “Learning Transferable Visual Models From Natural Language Supervision”

You can think of this as a 1-NN algorithm with the vectors as the training data



Radford et al “Learning Transferable Visual Models From Natural Language Supervision”

Since CLIP was trained with phrases, you can improve performance by using a phrase “A photo of a [category]”



+1.3% on ImageNet

Radford et al “Learning Transferable Visual Models From Natural Language Supervision”

A single phrase might be too biased.
Solution: Use multiple phrases



Image Encoder

Image vector

A photo of a plane | A drawing of a plane | …

A photo of a dog | A drawing of a dog | …

A photo of a bird | A drawing of a bird | …

Text Encoder

“Plane” vector 1	“Plane” vector 2
“Dog” vector 1	“Dog” vector 2
“Bird” vector 1	“Bird” vector 2

Radford et al “Learning Transferable Visual Models From Natural Language Supervision”

Use the average vector across phrases as the representation for each category



A photo of a plane | A drawing of a plane | …

A photo of a dog | A drawing of a dog | …

A photo of a bird | A drawing of a bird | …

Image Encoder

Text Encoder

+5% on ImageNet

Image vector

0.13

0.27

0.09

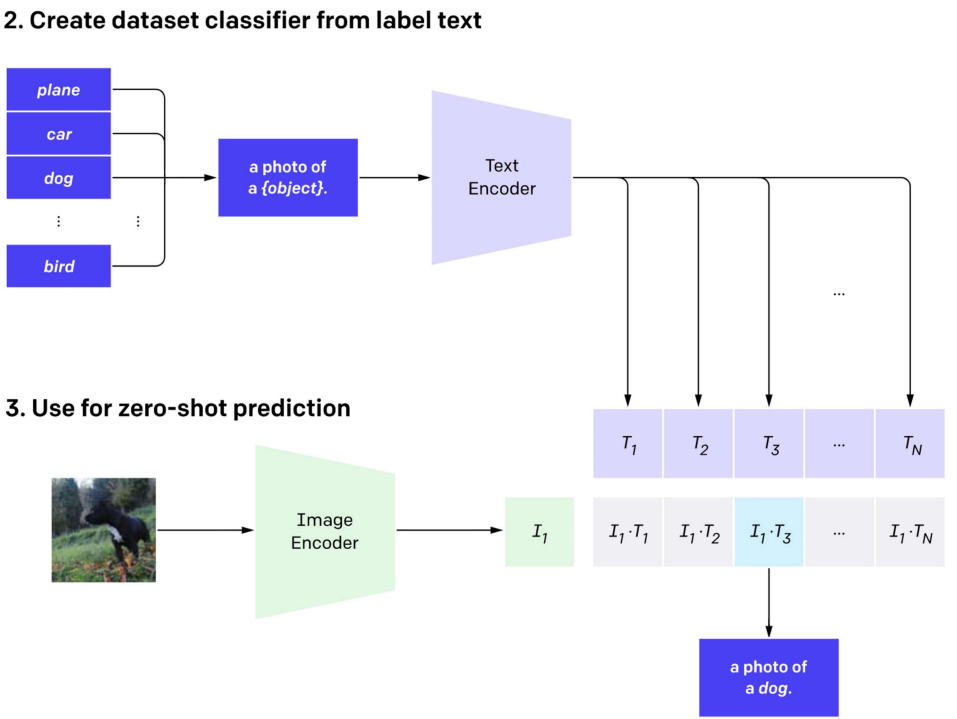
Mean “Plane” vector

Mean “Dog” vector

Mean “Bird” vector

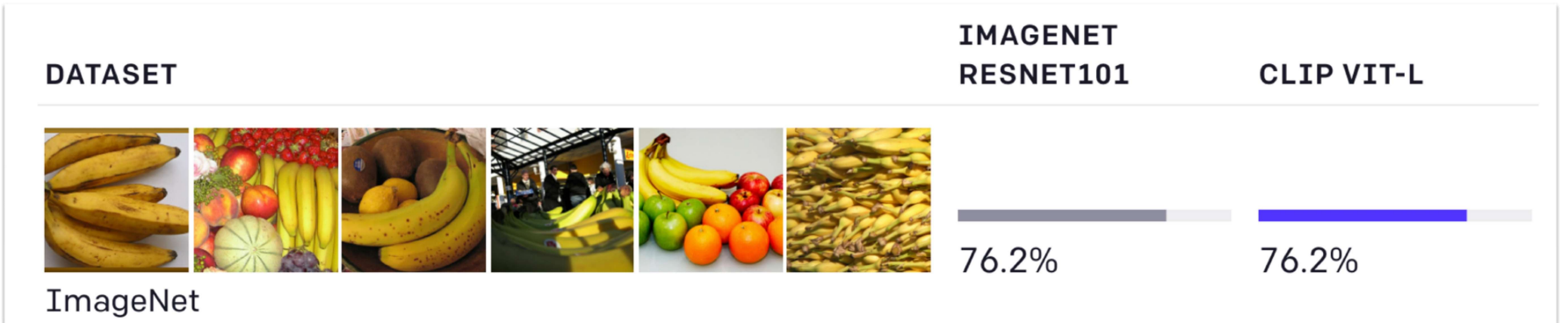
Radford et al “Learning Transferable Visual Models From Natural Language Supervision”

That's it! Now, you can use CLIP as a foundation model for image classification for any dataset



Radford et al "Learning Transferable Visual Models From Natural Language Supervision"

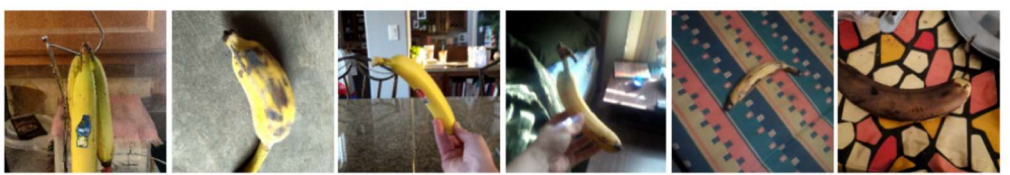
Exciting result after training on 400M image-text pairs



Matches the accuracy of of ResNet 101 that has been trained on ImageNet, except CLIP was trained with no human labels at all!

Radford et al “Learning Transferable Visual Models From Natural Language Supervision”

Here’s where things get even more exciting

DATASET	IMAGENET RESNET101	CLIP VIT-L
 ImageNet	76.2%	76.2%
 ObjectNet		

Radford et al “Learning Transferable Visual Models From Natural Language Supervision”

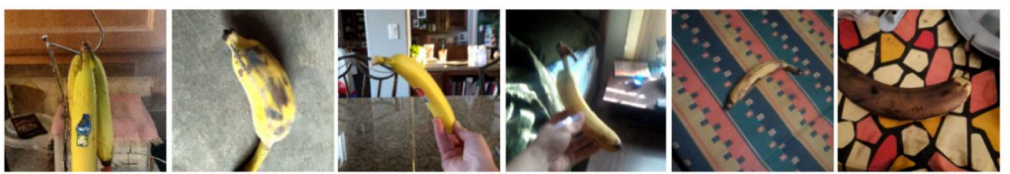
Training on ImageNet doesn't generalize to other datasets. ObjectNet contains the same categories but in weird viewpoints

DATASET	IMAGENET RESNET101	CLIP VIT-L
 ImageNet	76.2%	76.2%
 ObjectNet	32.6%	

Radford et al "Learning Transferable Visual Models From Natural Language Supervision"

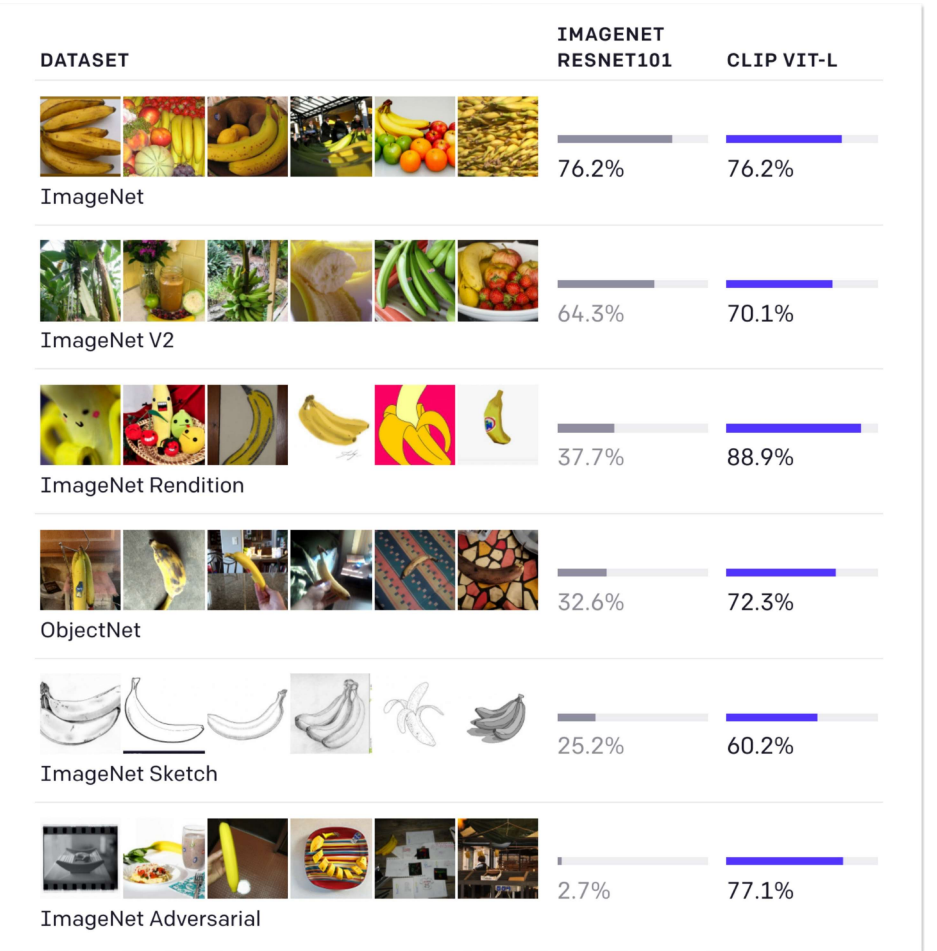
But CLIP zero-shot does so well!

Q. Why do you think that is?

DATASET	IMAGENET RESNET101	CLIP VIT-L
 ImageNet	76.2%	76.2%
 ObjectNet	32.6%	72.3%

Radford et al “Learning Transferable Visual Models From Natural Language Supervision”

CLIP performance is great also on graphic images , sketches, adversarial datasets,



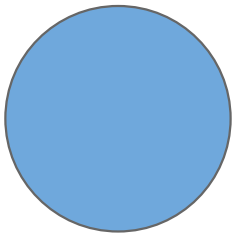
Radford et al “Learning Transferable Visual Models From Natural Language Supervision”

Why does CLIP perform so well?

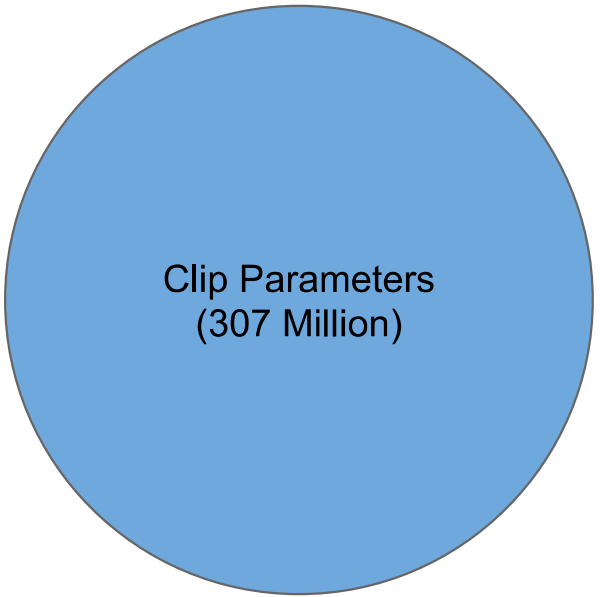
How can no labels beat labels??

Scale!

CLIP scaled up the model parameters with the transformer architecture



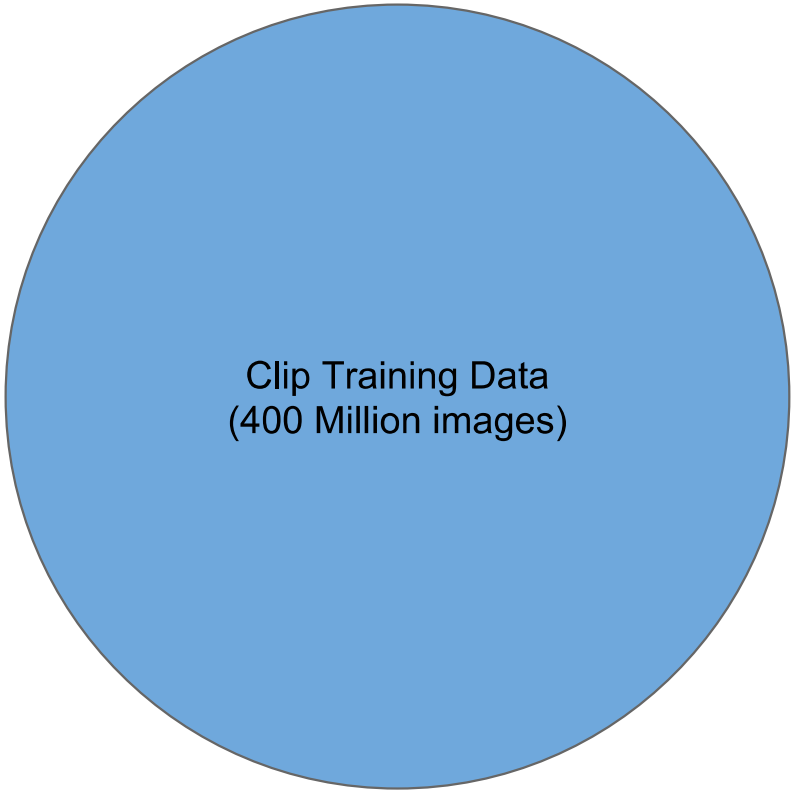
ImageNet ResNet Parameters
(44.5 Million)



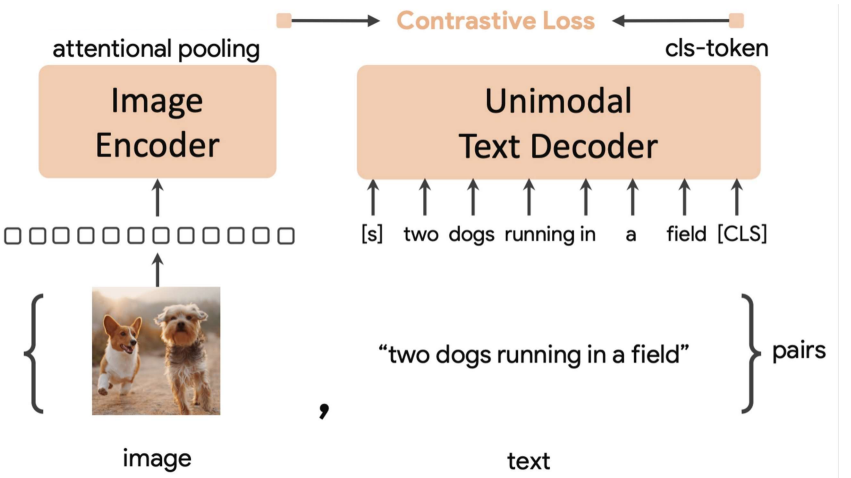
CLIP Scaled up the training data by scraping image-text pairs from the internet



ImageNet ResNet Training Data
(1.28 Million)

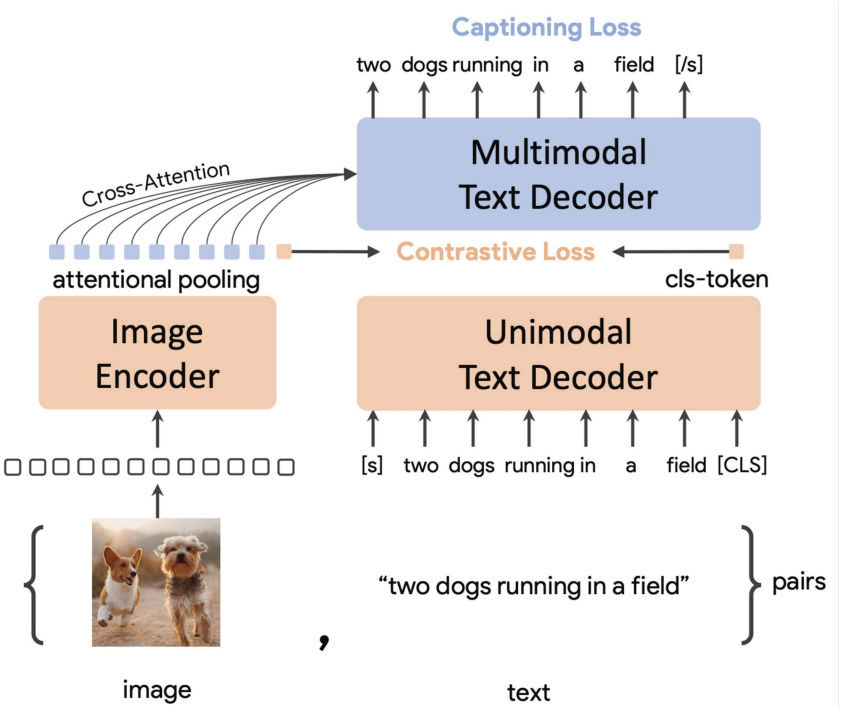


CoCa improved upon CLIP by adding a generation objective



"Contrastive Captioners are Image-Text Foundation Models", 2022

CoCa added a decoder with a captioning loss



"Contrastive Captioners are Image-Text Foundation Models", 2022

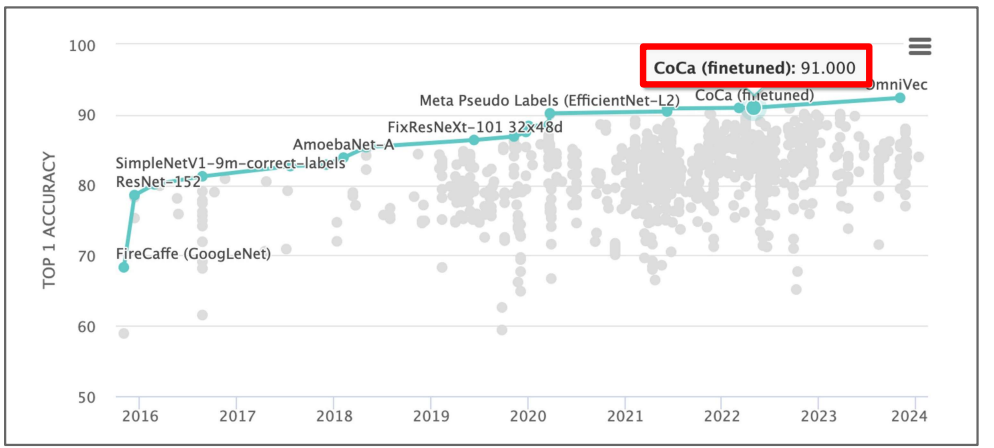
CoCa: Contrastive Captioners are Image-Text Foundation Models

Model	ImageNet	ImageNet-A	ImageNet-R	ImageNet-V2	ImageNet-Sketch	ObjectNet	Average
CLIP [12]	76.2	77.2	88.9	70.1	60.2	72.3	74.3
ALIGN [13]	76.4	75.8	92.2	70.1	64.8	72.2	74.5
FILIP [61]	78.3	-	-	-	-	-	-
Florence [14]	83.7	-	-	-	-	-	-
LiT [32]	84.5	79.4	93.9	78.7	-	81.1	-
BASIC [33]	85.7	85.6	95.7	80.6	76.1	78.9	83.7
CoCa-Base	82.6	76.4	93.2	76.5	71.7	71.6	78.7
CoCa-Large	84.8	85.7	95.6	79.6	75.7	78.6	83.3
CoCa	**86.3**	**90.2**	**96.5**	**80.7**	**77.6**	**82.7**	**85.7**

Table 4: Zero-shot image classification results on ImageNet [9], ImageNet-A [64], ImageNet-R [65], ImageNet-V2 [66], ImageNet-Sketch [67] and ObjectNet [68].

Classifier foundation models now beat all other models on ImageNet

Model	ImageNet
ALIGN [13]	88.6
Florence [14]	90.1
MetaPseudoLabels [51]	90.2
CoAtNet [10]	90.9
ViT-G [21]	90.5
+ Model Soups [52]	90.9
CoCa (frozen)	90.6
CoCa (finetuned)	**91.0**



Advantages of CLIP-style models

1. Dot product is super efficient
   a. Easy to train (enables scaling)
   b. Fast inference, e.g., retrieval over 5B images

2. Open-vocabulary (zero-shot generalization)

3. Can be chained with other models (CuPL)
   [we will discuss this later today]

April 2022, Tristan Thrush et al:

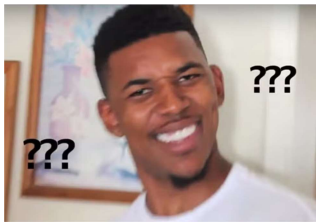
CLIP can’t distinguish between:



there is a mug in some grass



there is some grass in a mug



Disadvantages of CLIP-style models

1. Rely too heavily on batch size to learn concepts

Increasing batch size helps you understand fine-grained concepts



Batch size: 4	“animal”
Batch size: 100	“dog”
Batch size: **32000**	“Welsh Corgi”

Disadvantages of CLIP-style models

1. Rely too heavily on batch size to learn concepts

   Increasing batch size helps you understand fine-grained concepts

   But there's a limit to how fine-grained you can get this way

   Even in a batch of 32K, it's unlikely you see both "a mug in some grass" and "some grass in a mug"

Disadvantages of CLIP-style models

1. Rely too heavily on batch size to learn concepts

Winoground



there is a mug in some grass



there is some grass in a mug

“compositionality”

CREPE



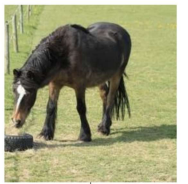
Crepe on a skillet. ?

Boats on a skillet.

Crepe under a skillet.

Crepe on a dog.

ARO



BLIP

the grass is eating the horse 81%

the horse is eating the grass 78%

…

Paper	Venue	Perturbation
Winoground	CVPR 2022 (Oral)	word order
VL-Checklist	EMNLP 2022	replacements
When-and-Why	ICLR 2023 (Oral)	word order
CREPE	CVPR 2023 (Spotlight)	word order replacements negations
SVLC	CVPR 2023	replacements
DAC	NeurIPS 2023 (spotlight)	replacements
What’s Up	EMNLP 2023	replacements
Text encoders…	EMNLP 2023	word order
SugarCREPE	NeurIPS 2023	word order replacements additions
COLA	NeurIPS 2023 D&B	replacements

Disadvantages of CLIP-style models

1. Rely too heavily on batch size to learn concepts

But training with hard negatives has its own problems…

A black cat and a brown dog
✓

A brown cat and a black dog
✗

A brown dog and a black cat
✗

“hard positives”

Disadvantages of CLIP-style models

1. Rely too heavily on batch size to learn concepts
2. Image-level captions are insufficient supervision

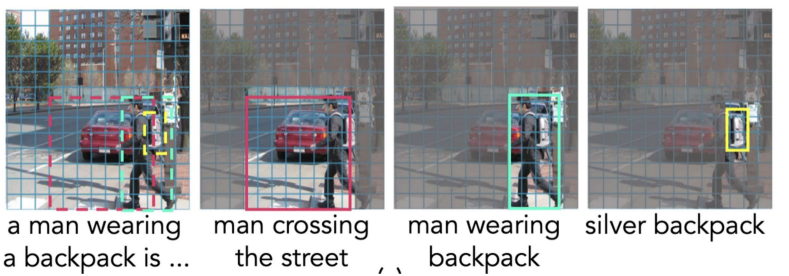


“living room”	✓
“house plants”	✗
“couch”	✗

Disadvantages of CLIP-style models

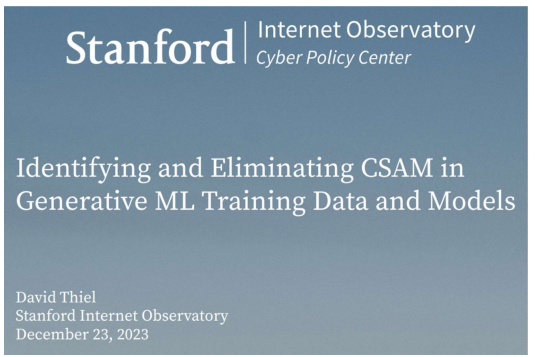
1. Rely too heavily on batch size to learn concepts
2. Image-level captions are insufficient supervision

Also train on region captions
with bounding box coordinates



Disadvantages of CLIP-style models

1. Rely too heavily on batch size to learn concepts
2. Image-level captions are insufficient supervision
3. You can't know everything in your 5B dataset



It's extremely important to be intentional about data collection and filtering

LLaVA

Motivation: Language models which do next token prediction can be applied to a wide variety of tasks at inference (Math, sentiment analysis, symbolic reasoning)

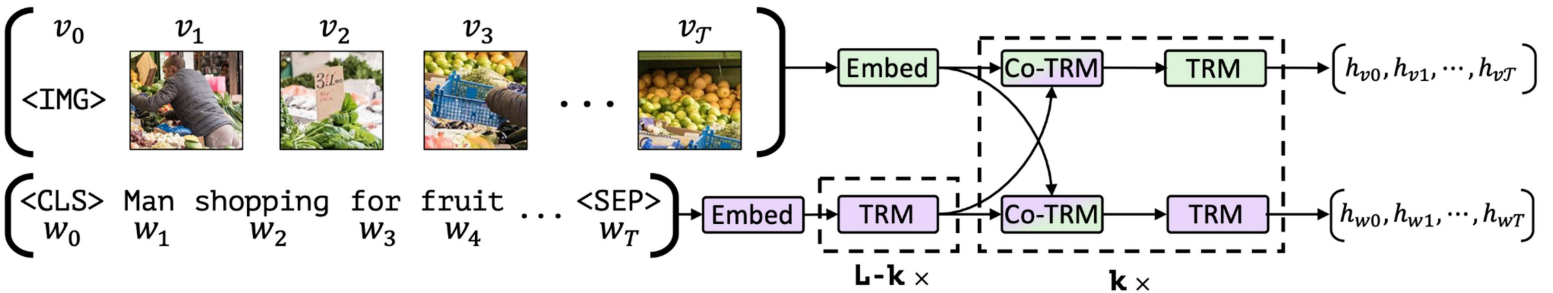
Can we build a model that can accept images and text as input, and then output text?

→ **Vision-Language Models**

First, some historical context

Vision-Language Models didn't start with LLaVA!

They go as far back as 2019 → ViLBERT



Historical context

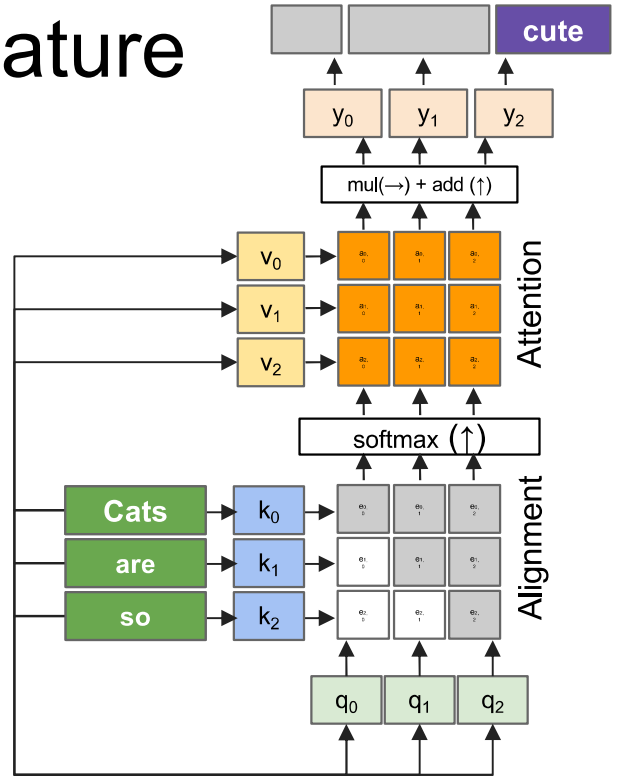
Vision-Language Models didn't start with LLaVA!

They go as far back as 2019 → ViLBERT

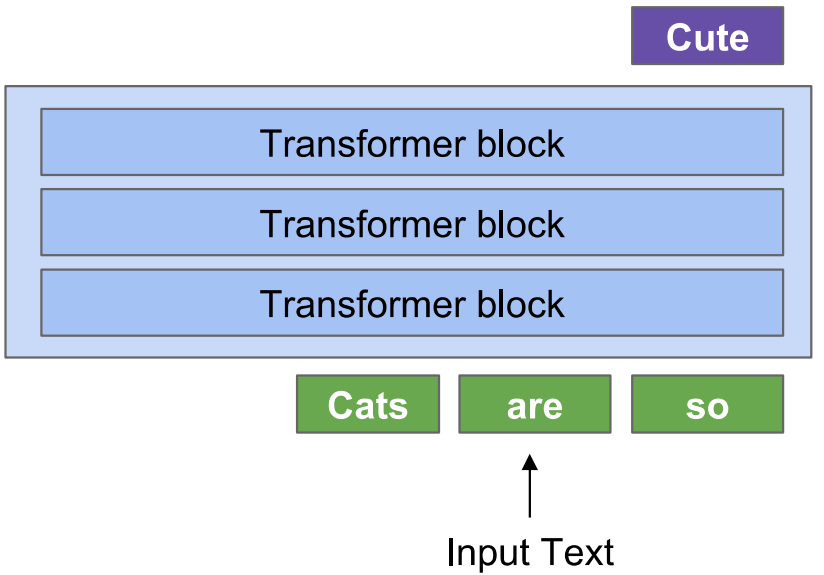
BUT, they had to finetune for each task separately,
with non-trivial task-specific methods (e.g., Mask-RCNN
bounding box re-ranking for RefCOCO)

→ Same paradigm as we discussed right at the beginning of this lecture:
very task-specific

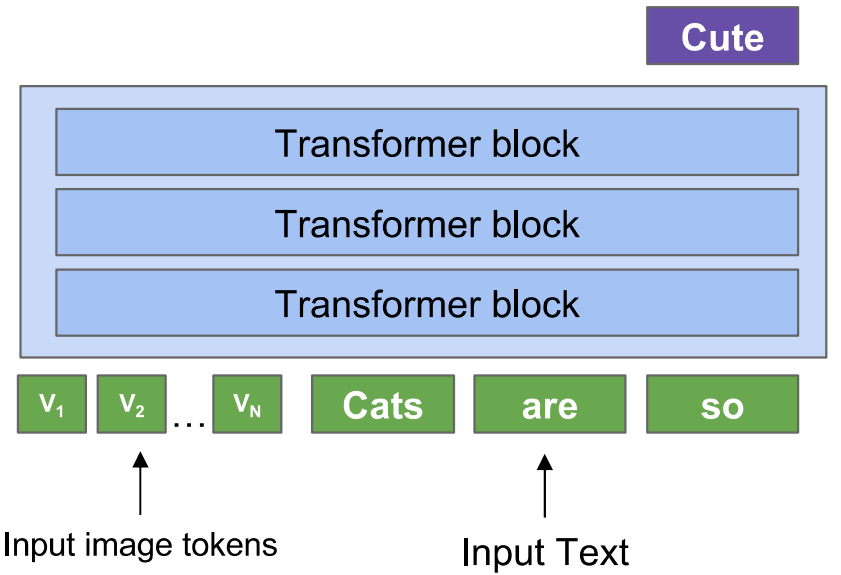
LLaVA uses the autoregressive nature of LLMs



Recall how transformers decode language

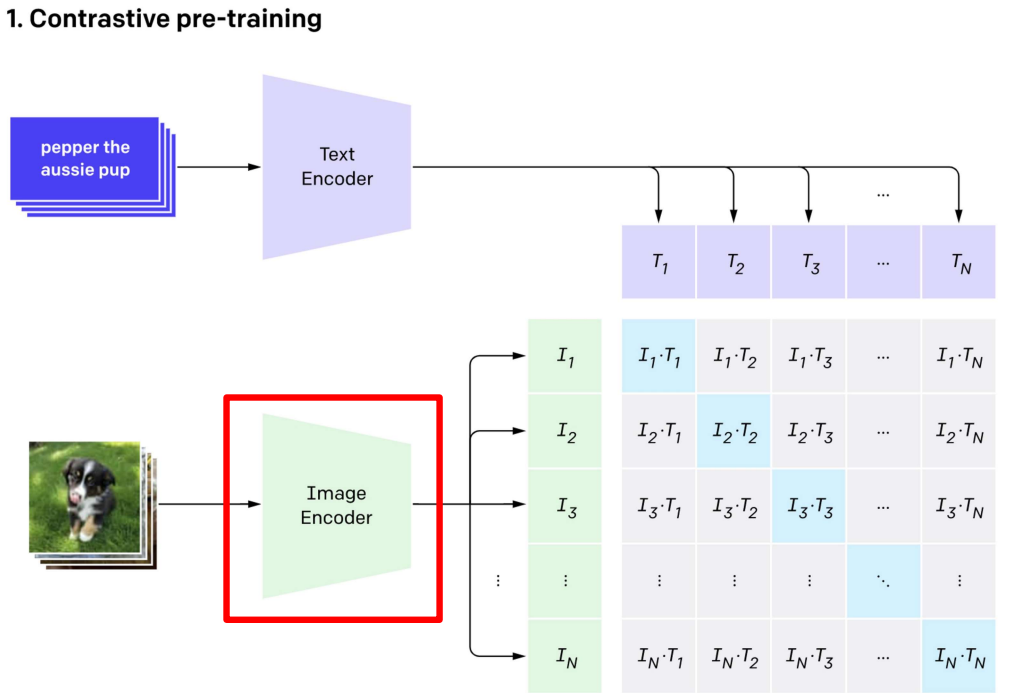


Key idea behind LLaVA – add visual information to the LLM



Which image tokens work best here?

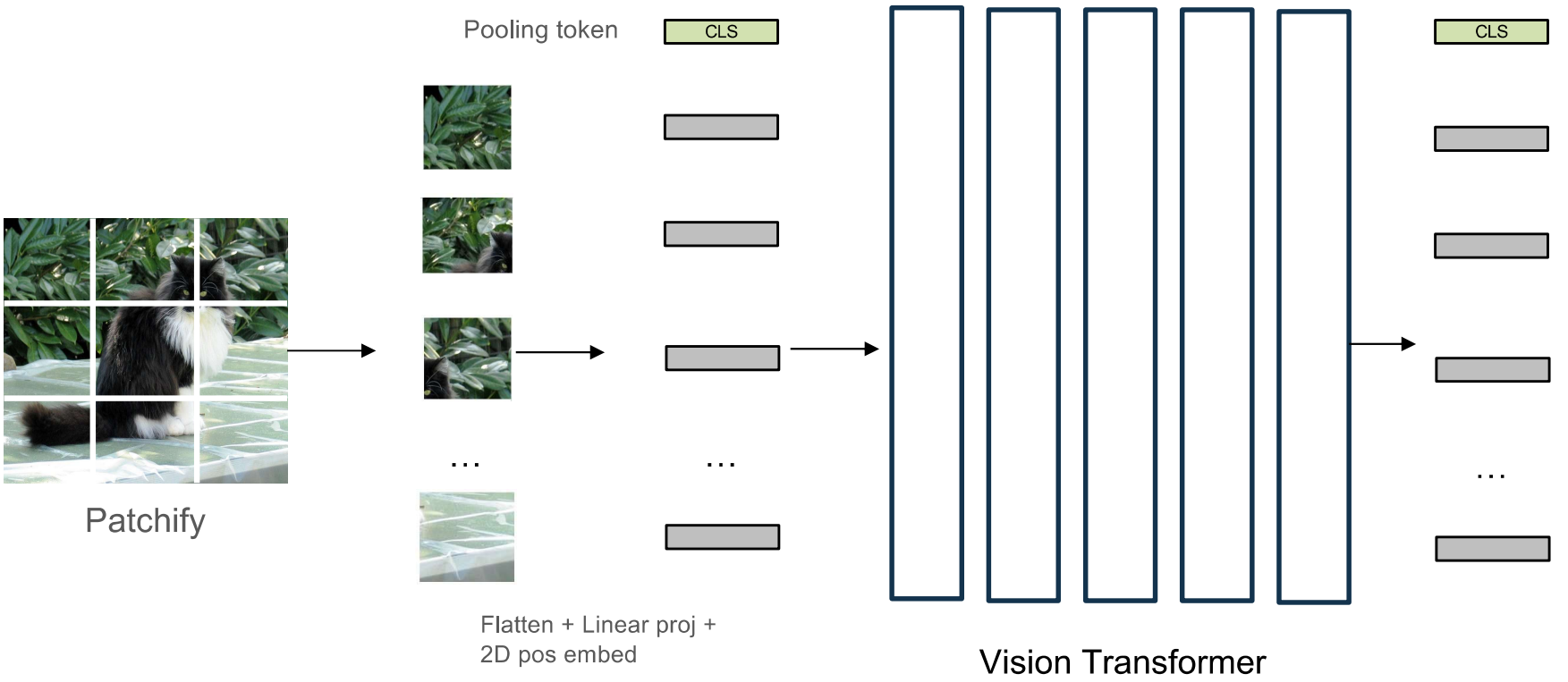
The CLIP encoder is a good option!



At the end of training, you have a model that will give you a similarity score between an image and a text

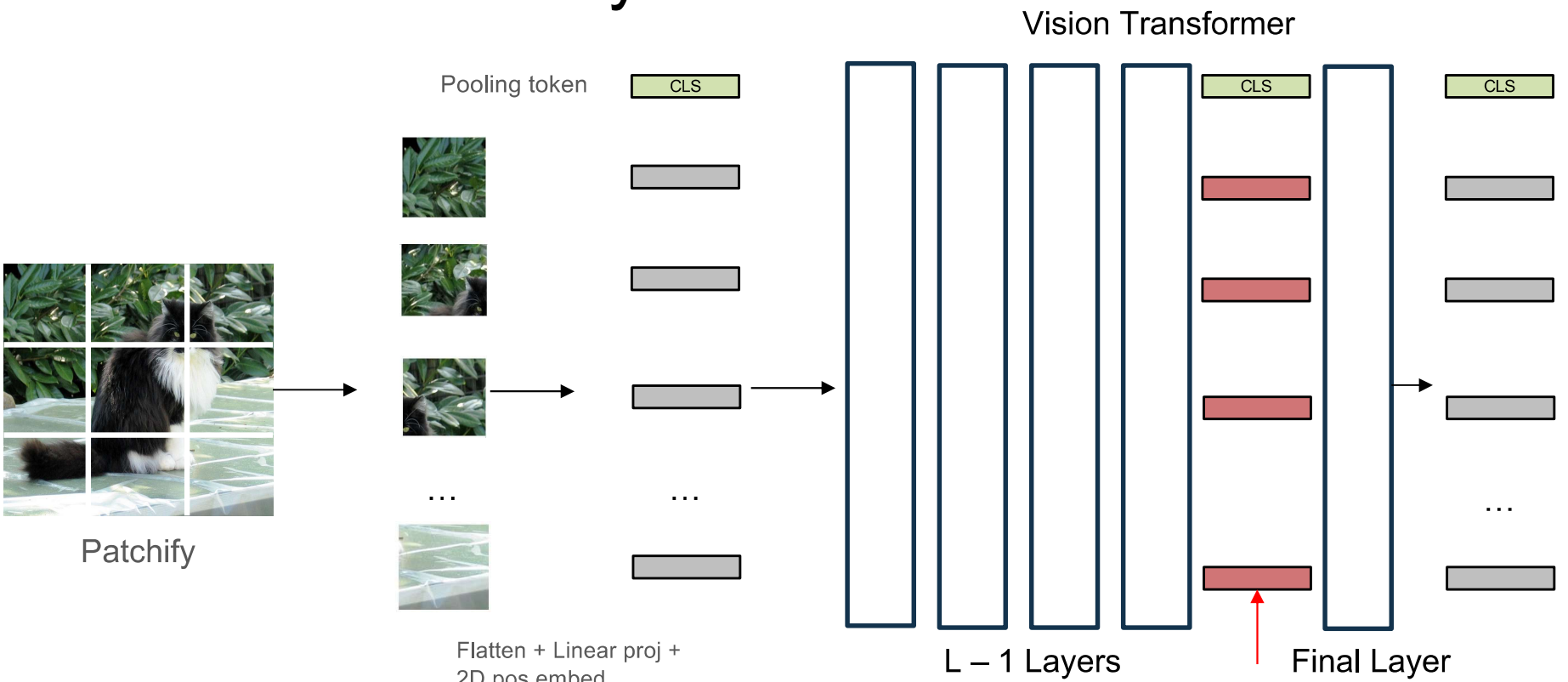
What features should we use from CLIP?

Extract Image Features from CLS token for contrastive learning



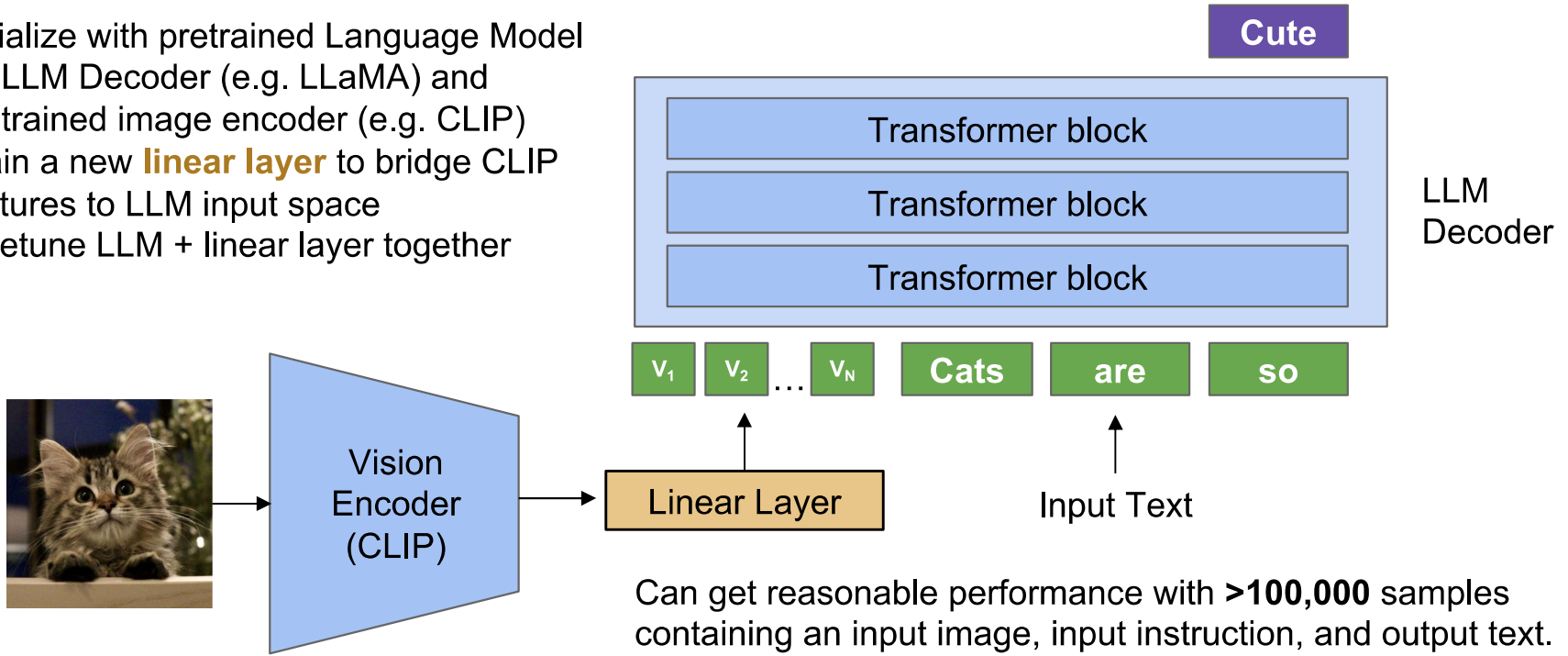
[Image source]

Use Penultimate Layer!



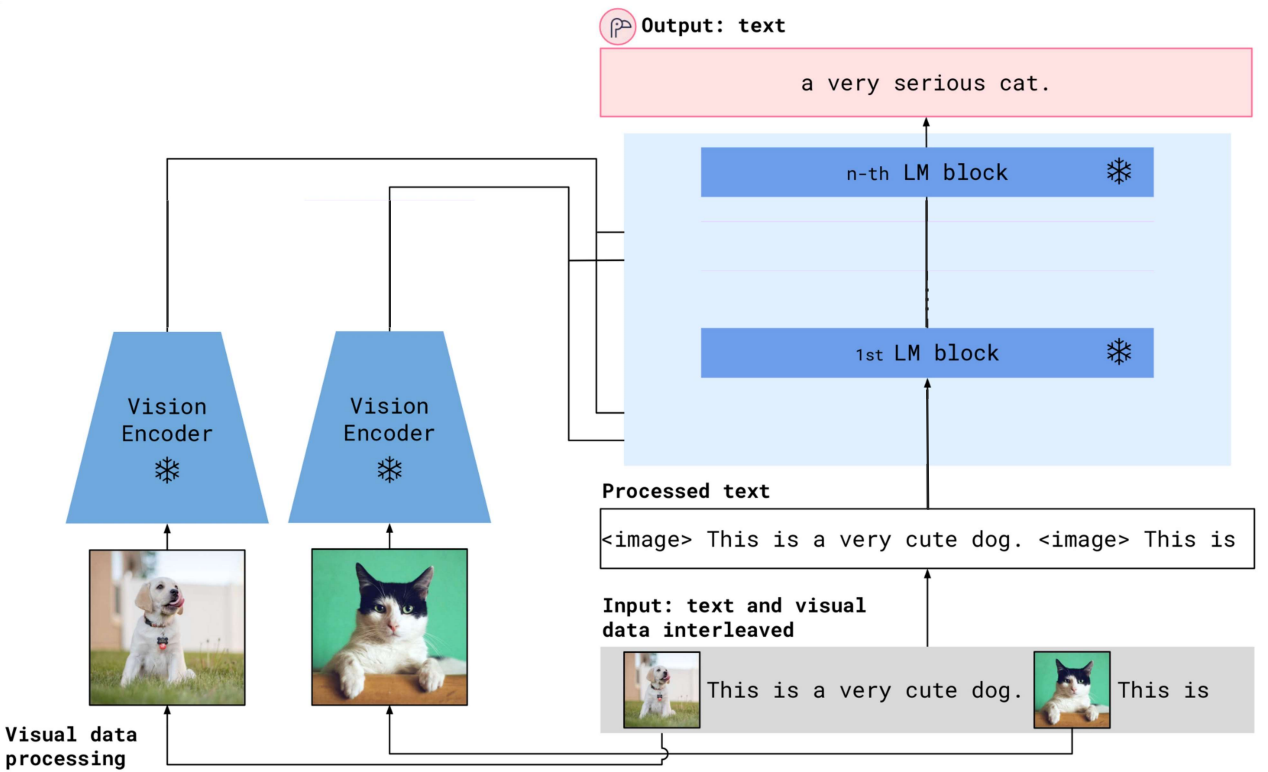
LLaVA – Overall Architecture + Training Recipe

1. Initialize with pretrained Language Model for LLM Decoder (e.g. LLaMA) and pretrained image encoder (e.g. CLIP)
2. Train a new **linear layer** to bridge CLIP features to LLM input space
3. Finetune LLM + linear layer together



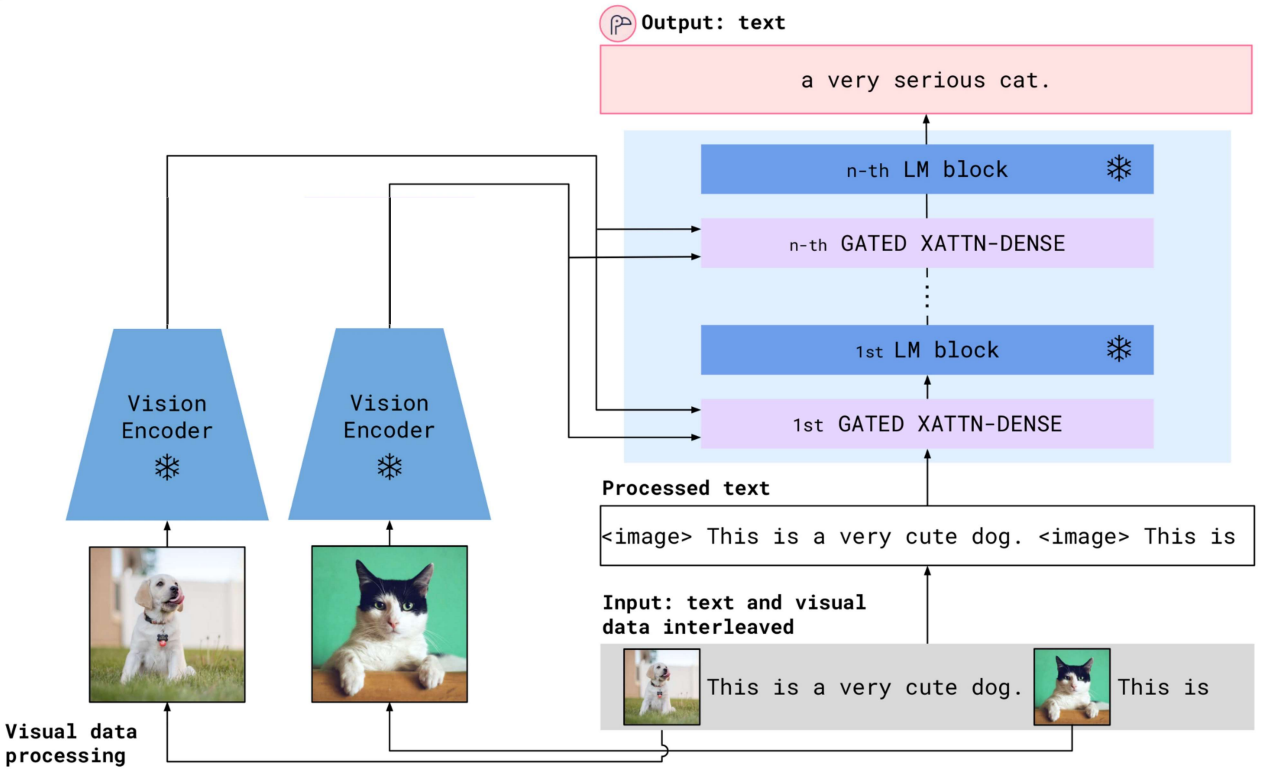
Can get reasonable performance with **>100,000** samples containing an input image, input instruction, and output text.

Pre-trained parts of Flamingo



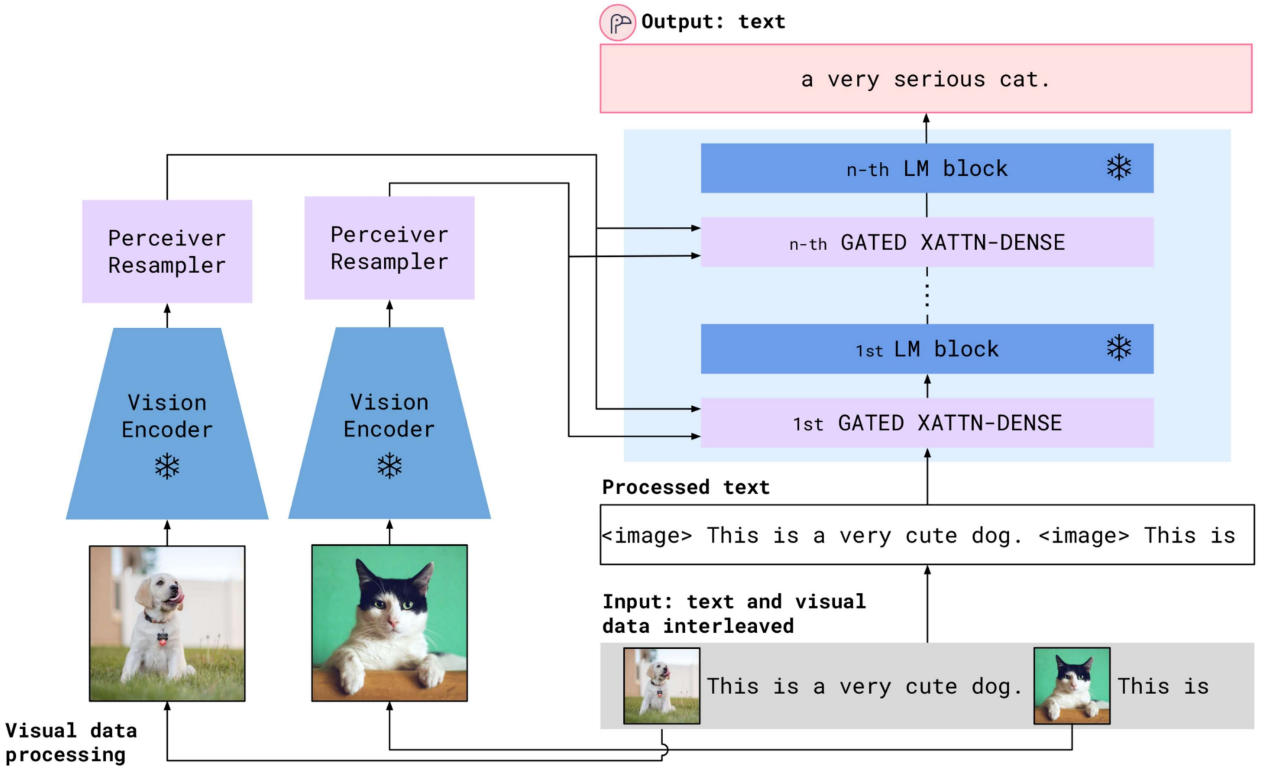
Alayrac et al “Flamingo: a Visual Language Model for Few-Shot Learning. 2022.

There are 2 learned parts in Flamingo



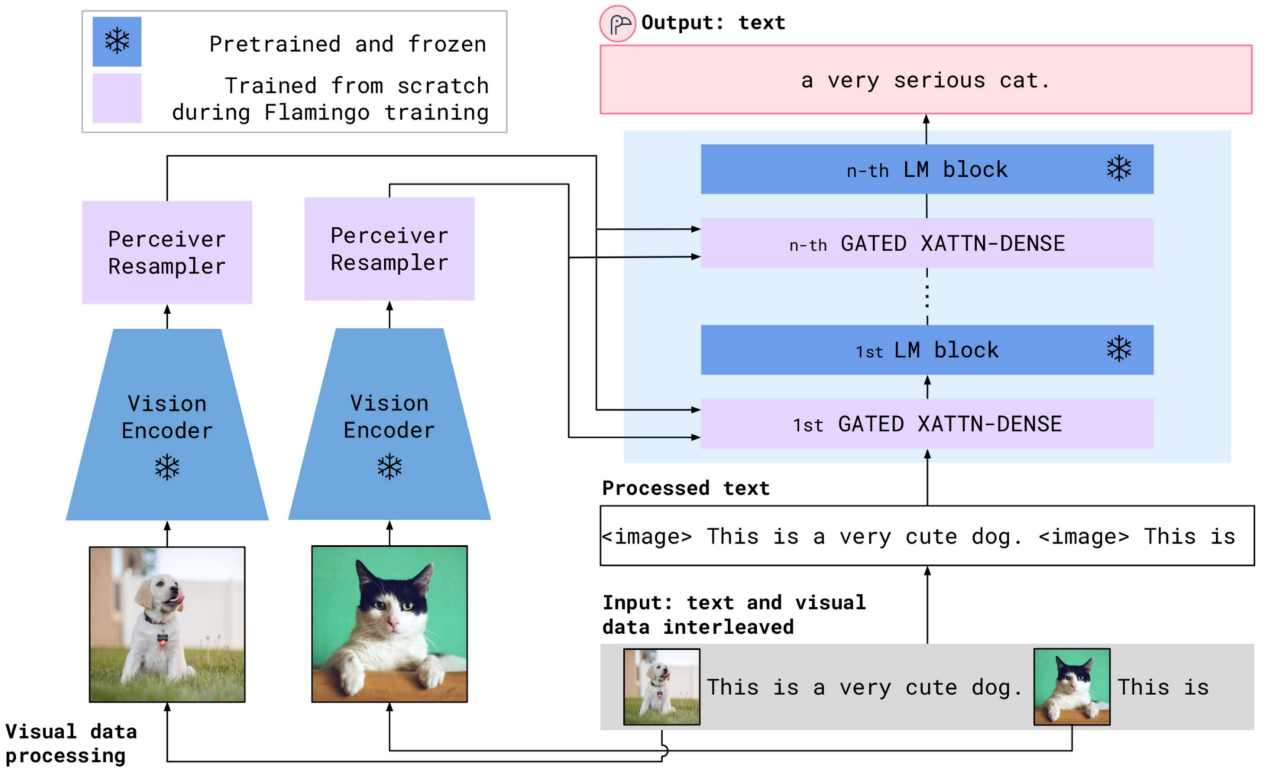
Alayrac et al "Flamingo: a Visual Language Model for Few-Shot Learning, 2022.

Perceiver sampler converts variable sized image tokens to fixed sized ones



Alayrac et al "Flamingo: a Visual Language Model for Few-Shot Learning, 2022.

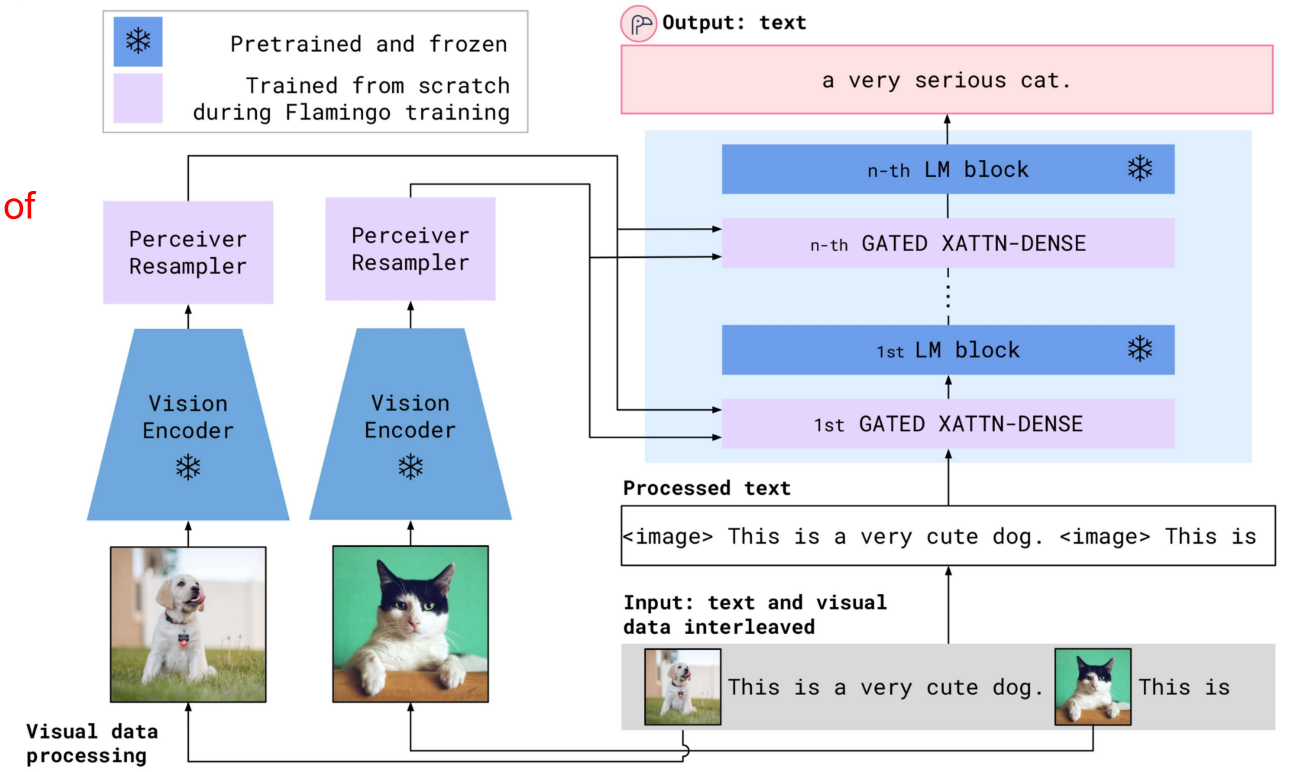
Flamingo full architecture



Alayrac et al "Flamingo: a Visual Language Model for Few-Shot Learning, 2022.

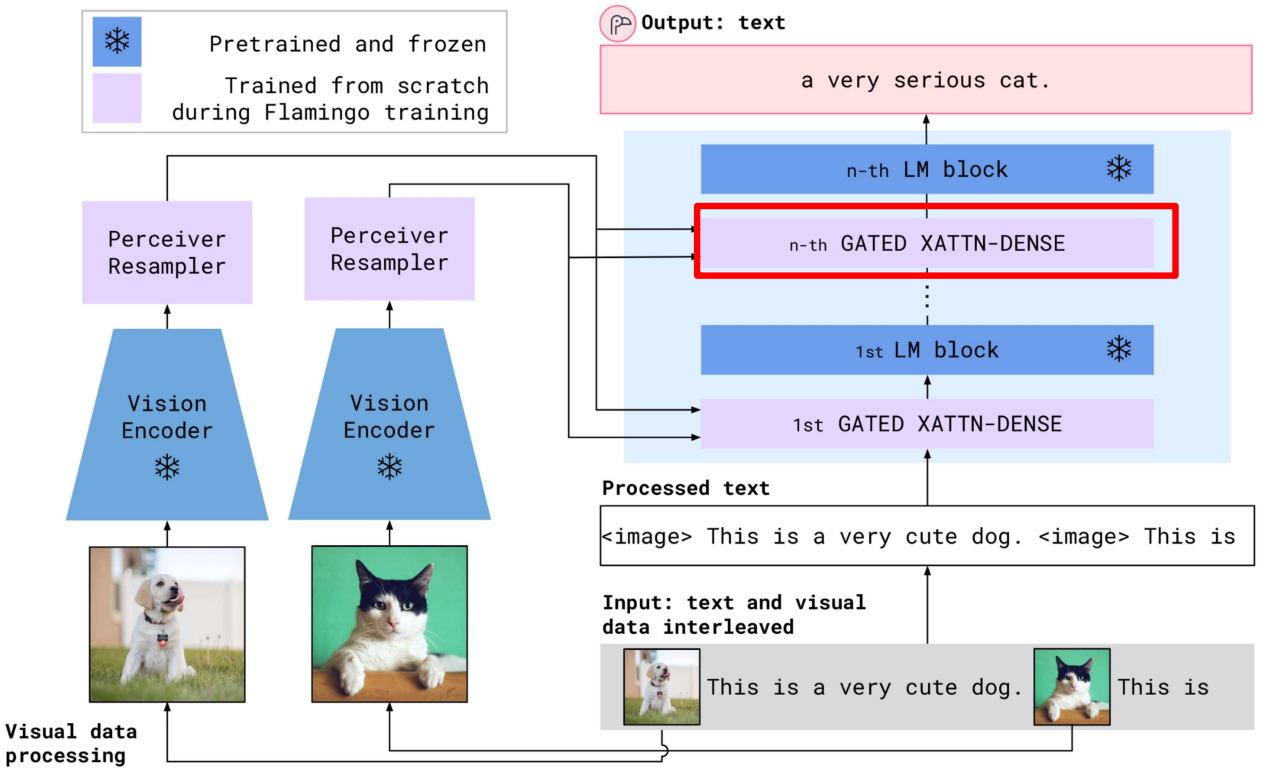
Flamingo full architecture

Learned method of down-sampling image/video representations



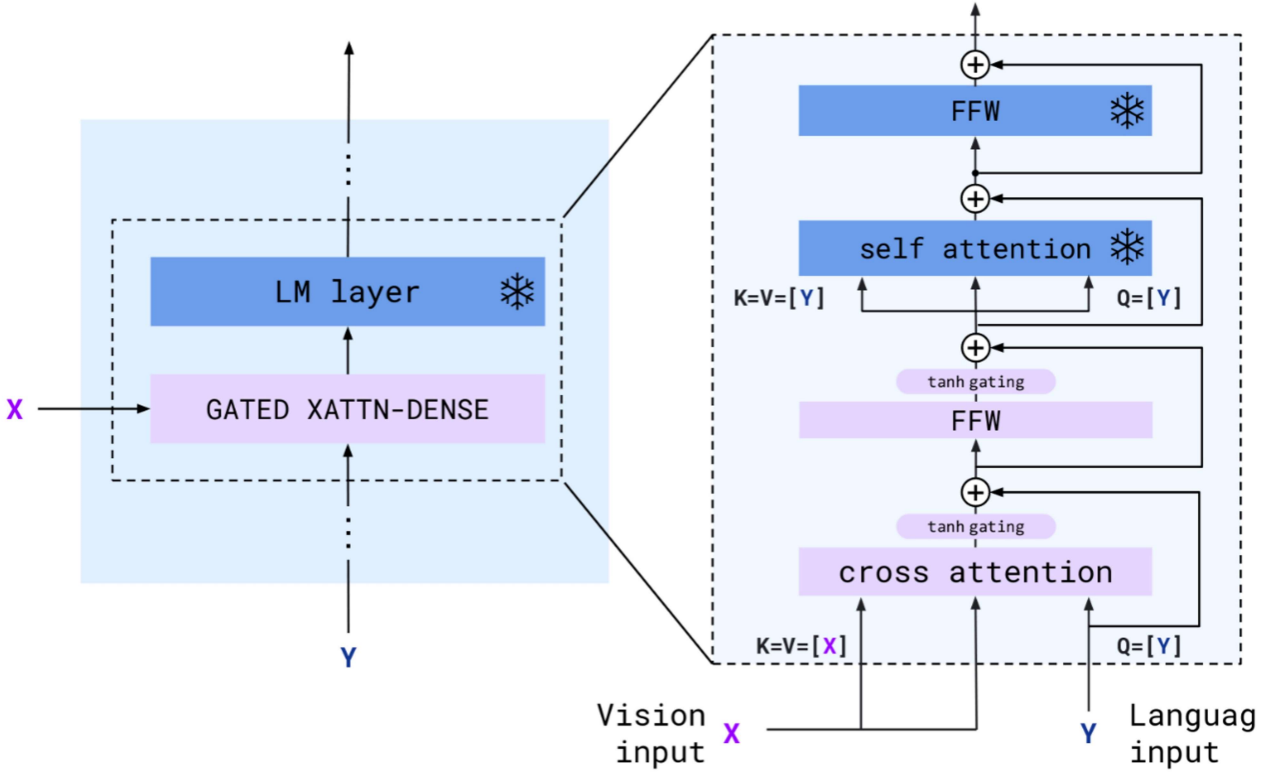
Alayrac et al "Flamingo: a Visual Language Model for Few-Shot Learning, 2022.

Flamingo full architecture



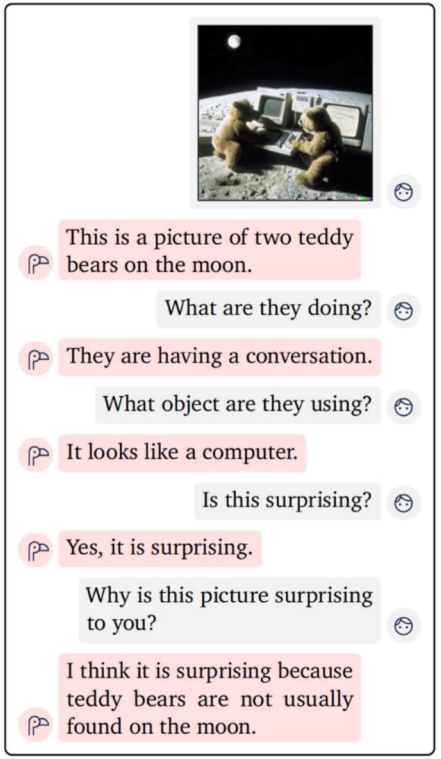
Alayrac et al "Flamingo: a Visual Language Model for Few-Shot Learning, 2022.

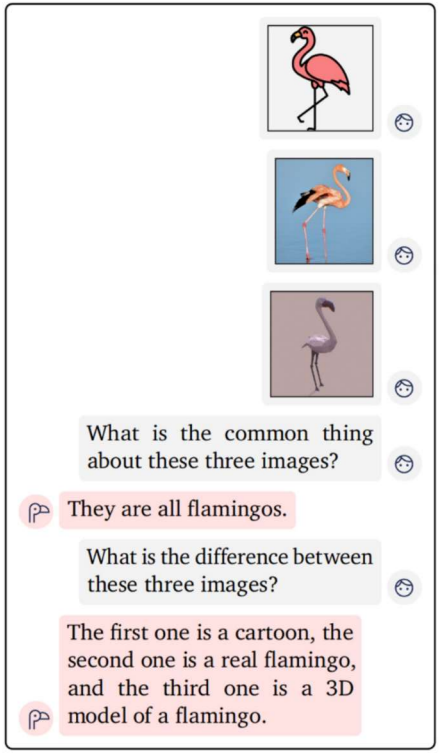
Flamingo gated cross-attention

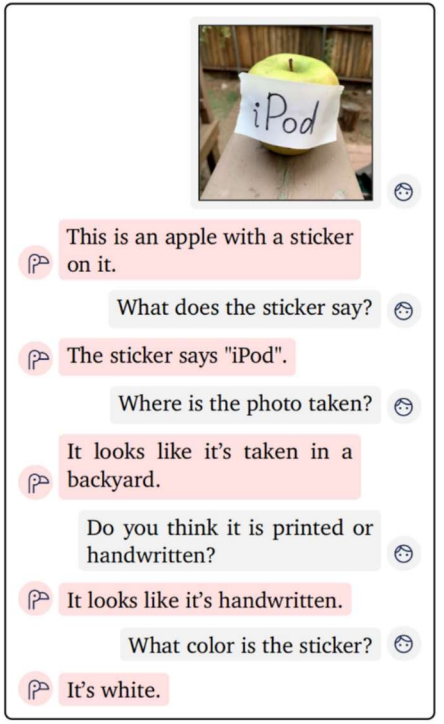


Alayrac et al "Flamingo: a Visual Language Model for Few-Shot Learning, 2022.

Flamingo results

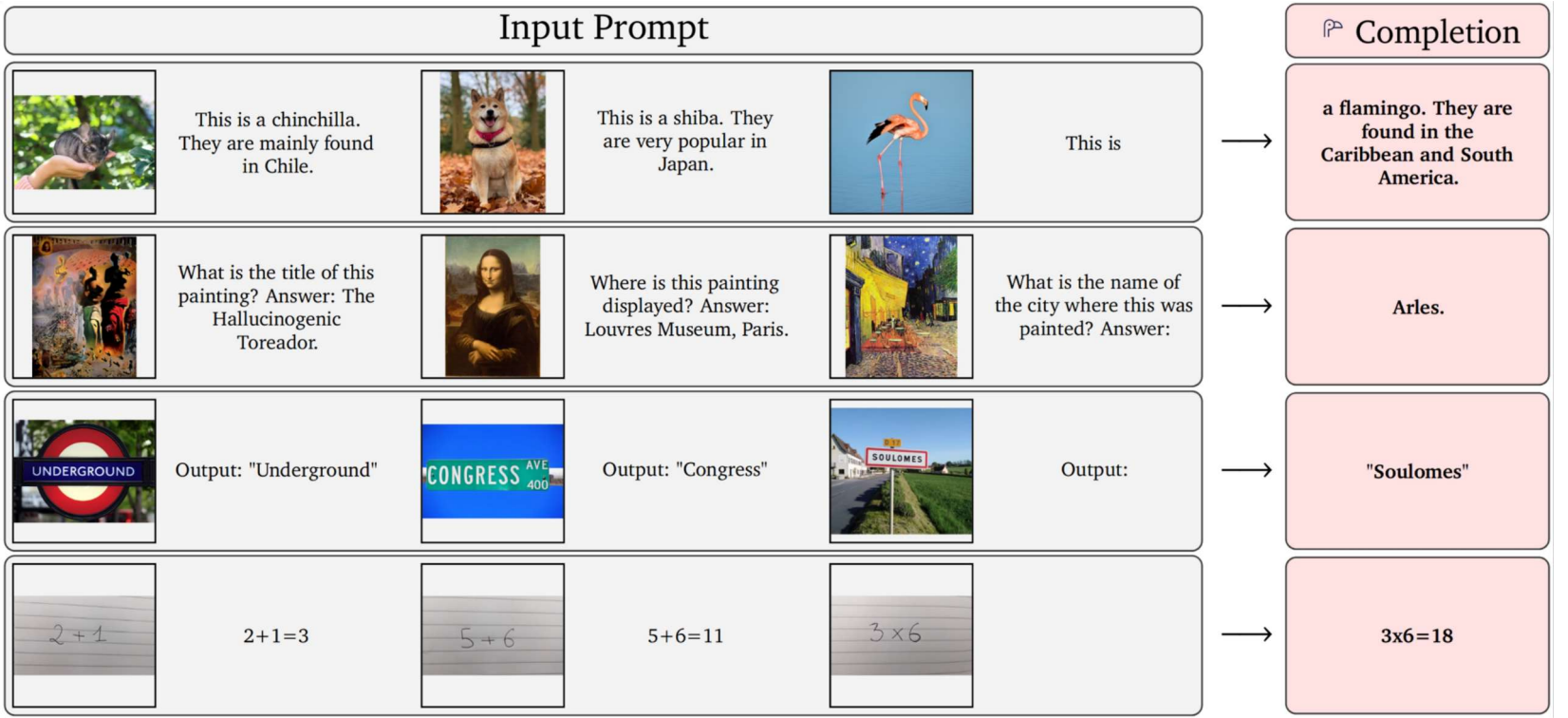






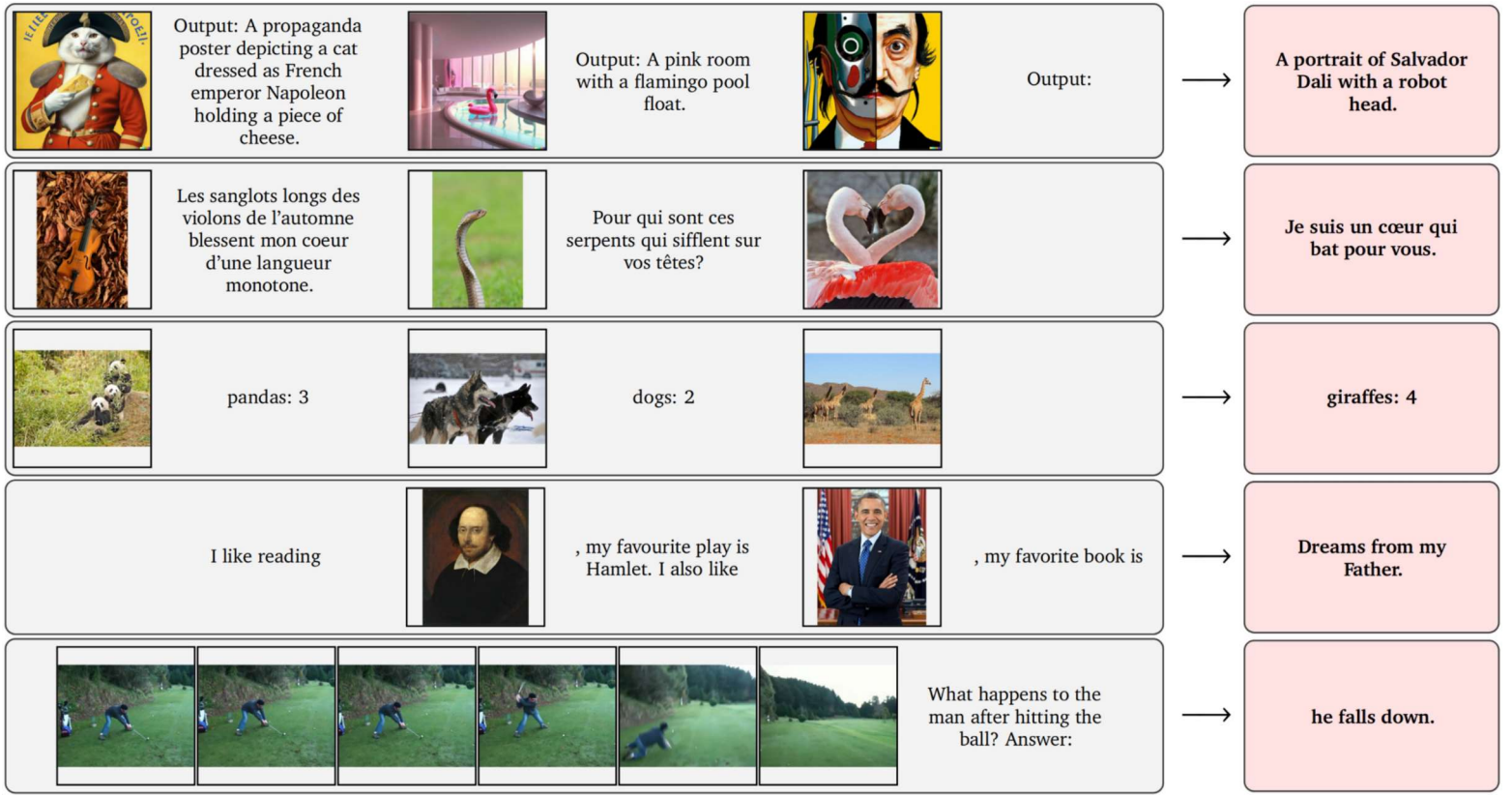
Alayrac et al "Flamingo: a Visual Language Model for Few-Shot Learning, 2022.

Flaming enables in-context learning



Alayrac et al "Flamingo: a Visual Language Model for Few-Shot Learning, 2022.

Flamingo results



Alayrac et al "Flamingo: a Visual Language Model for Few-Shot Learning, 2022.

Results: zero & few shot

Method	FT	Shot	OKVQA	VQAv2	COCO	MSVDQA	VATEX	VizWiz	Flick30K	MSRVTTQA	iVQA	YouCook2	STAR	VisDial	TextVQA	NextQA	HatefulMemes	RareAct
Zero/Few shot SOTA	✗	(X)	[39] 43.3 (16)	[124] 38.2 (4)	[134] 32.2 (0)	[64] 35.2 (0)	-	-	-	[64] 19.2 (0)	[145] 12.2 (0)	-	[153] 39.4 (0)	[87] 11.6 (0)	-	-	[94] 66.1 (0)	[94] 40.7 (0)
Flamingo-3B	✗	0	41.2	49.2	73.0	27.5	40.1	28.9	60.6	11.0	32.7	55.8	39.6	46.1	30.1	21.3	53.7	58.4
	✗	4	43.3	53.2	85.0	33.0	50.0	34.0	72.0	14.9	35.7	64.6	41.3	47.3	32.7	22.4	53.6	-
	✗	8	44.6	55.4	90.6	37.0	54.5	38.4	71.7	19.6	36.8	68.0	40.6	47.6	32.4	23.9	54.7	-
	✗	16	45.6	56.7	95.4	40.2	57.1	43.3	73.4	23.4	37.4	73.2	40.1	47.5	31.8	25.2	55.3	-
	✗	32	45.9	57.1	99.0	42.6	59.2	45.5	71.2	25.6	37.7	76.7	41.6	OOC	30.6	26.1	56.3	-
Flamingo-9B	✗	0	44.7	51.8	79.4	30.2	39.5	28.8	61.5	13.7	35.2	55.0	41.8	48.0	31.8	23.0	57.0	57.9
	✗	4	49.3	56.3	93.1	36.2	51.7	34.9	72.6	18.2	37.7	70.8	42.8	50.4	33.6	24.7	62.7	-
	✗	8	50.0	58.0	99.0	40.8	55.2	39.4	73.4	23.9	40.0	75.0	**43.4**	51.2	33.6	25.8	63.9	-
	✗	16	50.8	59.4	102.2	44.5	58.5	43.0	72.7	27.6	41.5	77.2	42.4	51.3	33.5	27.6	64.5	-
	✗	32	51.0	60.4	106.3	47.2	57.4	44.0	72.8	29.4	40.7	77.3	41.2	OOC	32.6	28.4	63.5	-
Flamingo	✗	0	50.6	56.3	84.3	35.6	46.7	31.6	67.2	17.4	40.7	60.1	39.7	52.0	35.0	26.7	46.4	**60.8**
	✗	4	57.4	63.1	103.2	41.7	56.0	39.6	75.1	23.9	44.1	74.5	42.4	55.6	36.5	30.8	68.6	-
	✗	8	57.5	65.6	108.8	45.5	60.6	44.8	78.2	27.6	44.8	80.7	42.3	56.4	37.3	32.3	**70.0**	-
	✗	16	57.8	66.8	110.5	48.4	62.8	48.4	**78.9**	30.0	45.2	84.2	41.1	**56.8**	37.6	32.9	**70.0**	-
	✗	32	**57.8**	**67.6**	**113.8**	**52.3**	**65.1**	**49.8**	75.4	**31.0**	**45.3**	**86.8**	42.2	OOC	**37.9**	**33.5**	**70.0**	-
Pretrained FT SOTA	✔	(X)	54.4 [39] (10K)	80.2 [150] (444K)	143.3 [134] (500K)	47.9 [32] (27K)	76.3 [165] (500K)	57.2 [70] (20K)	67.4 [162] (30K)	46.8 [57] (130K)	35.4 [145] (6K)	138.7 [142] (10K)	36.7 [138] (46K)	75.2 [87] (123K)	54.7 [147] (20K)	25.2 [139] (38K)	75.4 [60] (9K)	-

Results: zero & few shot

Method	FT	Shot	OKVQA	VQAv2	COCO	MSVDQA	VATEX	VizWiz	Flick30K	MSRVTTQA	iVQA	YouCook2	STAR	VisDial	TextVQA	NextQA	HatefulMemes	RareAct
Zero/Few shot SOTA	✗	(X)	[39] 43.3 (16)	[124] 38.2 (4)	[134] 32.2 (0)	[64] 35.2 (0)	-	-	-	[64] 19.2 (0)	[145] 12.2 (0)	-	[153] 39.4 (0)	[87] 11.6 (0)	-	-	[94] 66.1 (0)	[94] 40.7 (0)
Flamingo-3B	✗	0	41.2	49.2	73.0	27.5	40.1	28.9	60.6	11.0	32.7	55.8	39.6	46.1	30.1	21.3	53.7	58.4
	✗	4	43.3	53.2	85.0	33.0	50.0	34.0	72.0	14.9	35.7	64.6	41.3	47.3	32.7	22.4	53.6	-
	✗	8	44.6	55.4	90.6	37.0	54.5	38.4	71.7	19.6	36.8	68.0	40.6	47.6	32.4	23.9	54.7	-
	✗	16	45.6	56.7	95.4	40.2	57.1	43.3	73.4	23.4	37.4	73.2	40.1	47.5	31.8	25.2	55.3	-
	✗	32	45.9	57.1	99.0	42.6	59.2	45.5	71.2	25.6	37.7	76.7	41.6	OOC	30.6	26.1	56.3	-
Flamingo-9B	✗	0	44.7	51.8	79.4	30.2	39.5	28.8	61.5	13.7	35.2	55.0	41.8	48.0	31.8	23.0	57.0	57.9
	✗	4	49.3	56.3	93.1	36.2	51.7	34.9	72.6	18.2	37.7	70.8	42.8	50.4	33.6	24.7	62.7	-
	✗	8	50.0	58.0	99.0	40.8	55.2	39.4	73.4	23.9	40.0	75.0	**43.4**	51.2	33.6	25.8	63.9	-
	✗	16	50.8	59.4	102.2	44.5	58.5	43.0	72.7	27.6	41.5	77.2	42.4	51.3	33.5	27.6	64.5	-
	✗	32	51.0	60.4	106.3	47.2	57.4	44.0	72.8	29.4	40.7	77.3	41.2	OOC	32.6	28.4	63.5	-
Flamingo	✗	0	50.6	56.3	84.3	35.6	46.7	31.6	67.2	17.4	40.7	60.1	39.7	52.0	35.0	26.7	46.4	**60.8**
	✗	4	57.4	63.1	103.2	41.7	56.0	39.6	75.1	23.9	44.1	74.5	42.4	55.6	36.5	30.8	68.6	-
	✗	8	57.5	65.6	108.8	45.5	60.6	44.8	78.2	27.6	44.8	80.7	42.3	56.4	37.3	32.3	**70.0**	-
	✗	16	57.8	66.8	110.5	48.4	62.8	48.4	**78.9**	30.0	45.2	84.2	41.1	**56.8**	37.6	32.9	**70.0**	-
	✗	32	**57.8**	**67.6**	**113.8**	**52.3**	**65.1**	**49.8**	75.4	**31.0**	**45.3**	**86.8**	42.2	OOC	**37.9**	**33.5**	**70.0**	-
Pretrained FT SOTA	✔	(X)	54.4 [39] (10K)	80.2 [150] (444K)	143.3 [134] (500K)	47.9 [32] (27K)	76.3 [165] (500K)	57.2 [70] (20K)	67.4 [162] (30K)	46.8 [57] (130K)	35.4 [145] (6K)	138.7 [142] (10K)	36.7 [138] (46K)	75.2 [87] (123K)	54.7 [147] (20K)	25.2 [139] (38K)	75.4 [60] (9K)	-